

Card_PWA — Frontend-Architektur

Vollständige Analyse der React/TypeScript-PWA: Module, Komponenten, Hooks, Services, Datenmodelle, Speicherzugriffe und Abhängigkeiten — ausschließlich aus dem Quellcode abgeleitet. Diagramme in Standard-Mermaid-Syntax (flowchart / classDiagram / sequenceDiagram), kompatibel mit draw.io-Import.

~83.400

ZEILEN CODE

74

KOMponentEN

37

HOOKS

24

SERVICES

~66

UTILS-MODULE

3

DEXIE-DBS

12

DATENDATEIEN

0

ROUTER (MANUELL)

50 Gesamtübersicht

Kompakter Einstieg, bevor die folgenden Abschnitte in Module, Datenflüsse und Abhängigkeiten im Detail einsteigen.

Card_PWA ist eine Offline-first React-18/TypeScript-PWA (Vite-Build) zum Lernen von Prüfungsinhalten (CompTIA SY0-701) mit Karteikarten, Videos, Lab-Szenarien und einem eigenen SY0-701-„Lerneinheiten“-System. Es gibt **keinen Client-Router** — `App.tsx` hält den aktuell sichtbaren Screen als einfachen `useState<View>('home' | 'study' | 'shuffle-study' | 'shuffle-manage' | 'videos')` und rendert bedingt. Die Datenhaltung erfolgt vollständig lokal in drei getrennten **Dexie/IndexedDB**-Datenbanken; ein optionaler **eigener Sync-Server** (Raspberry Pi) synchronisiert per Operations-Log (Push-Queue + Pull-Cursor) zwischen Geräten. Ein handgeschriebener `service-worker.js` (1559 Zeilen, kein Workbox/vite-plugin-pwa) übernimmt App-Shell-Caching, Offline-Fallback, Background-/Periodic-Sync, Web-Push und einen Message-Bus zur App.

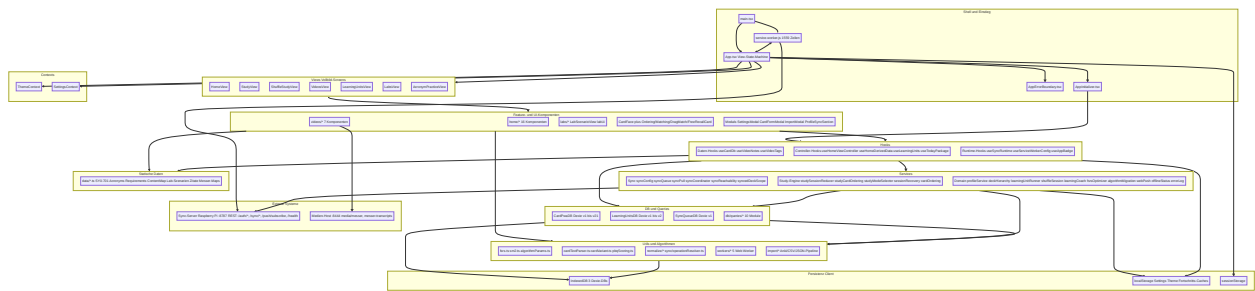
Architektonisch gliedert sich der Code in acht Schichten: **Shell/Views** (App.tsx + fünf Vollbild-Views) → **Feature- und UI-Komponenten** (components/*) → **Hooks** (Daten-Hooks mit Dexie liveQuery, Controller-Hooks für Home/Lerneinheiten, Runtime-Hooks) → **Services** (Sync-Protokoll, Study-Session-Engine, Domänenlogik) → **DB/Queries** (Dexie-Zugriffsschicht) → **Utils/Algorithmen** (FSRS/SM2, Parser, Normalisierung, Web Worker, Import-Pipeline) → **statische Daten** (data/*.ts) → **Persistenz** (IndexedDB, localStorage, sessionStorage). Zustandsverwaltung erfolgt bewusst ohne globalen Store (kein Redux/Zustand) — stattdessen React Context (Theme, Settings) für App-weite Einstellungen, lokaler Komponentenstate/useReducer für Sessions, und Dexie liveQuery als reaktive „Datenbank-als-Store“-Schicht, invalidiert zusätzlich über ein globales window-Event (REVIEW_UPDATED_EVENT).

§1 Architekturdiagramm

Schichten und Hauptmodule mit Import-/Nutzungsrichtung. Pfeile zeigen „nutzt/importiert“.

BLATT 01 · SCHICHTENARCHITEKTUR

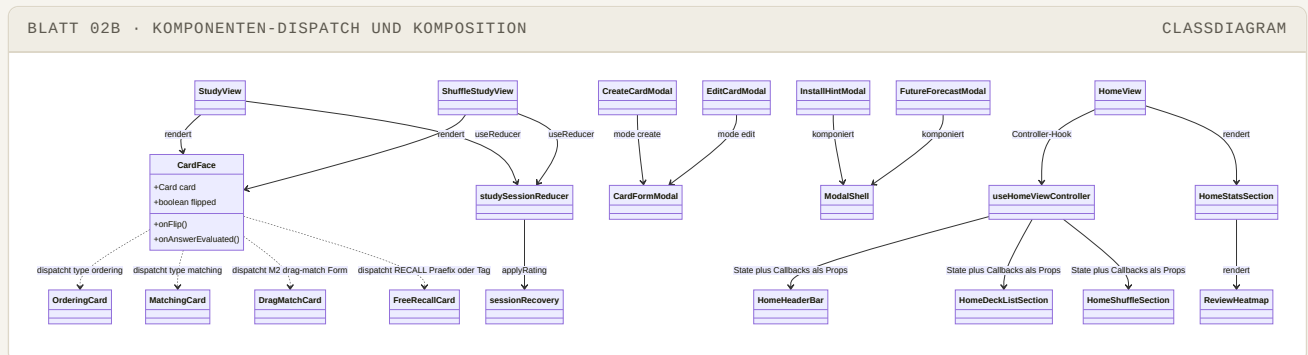
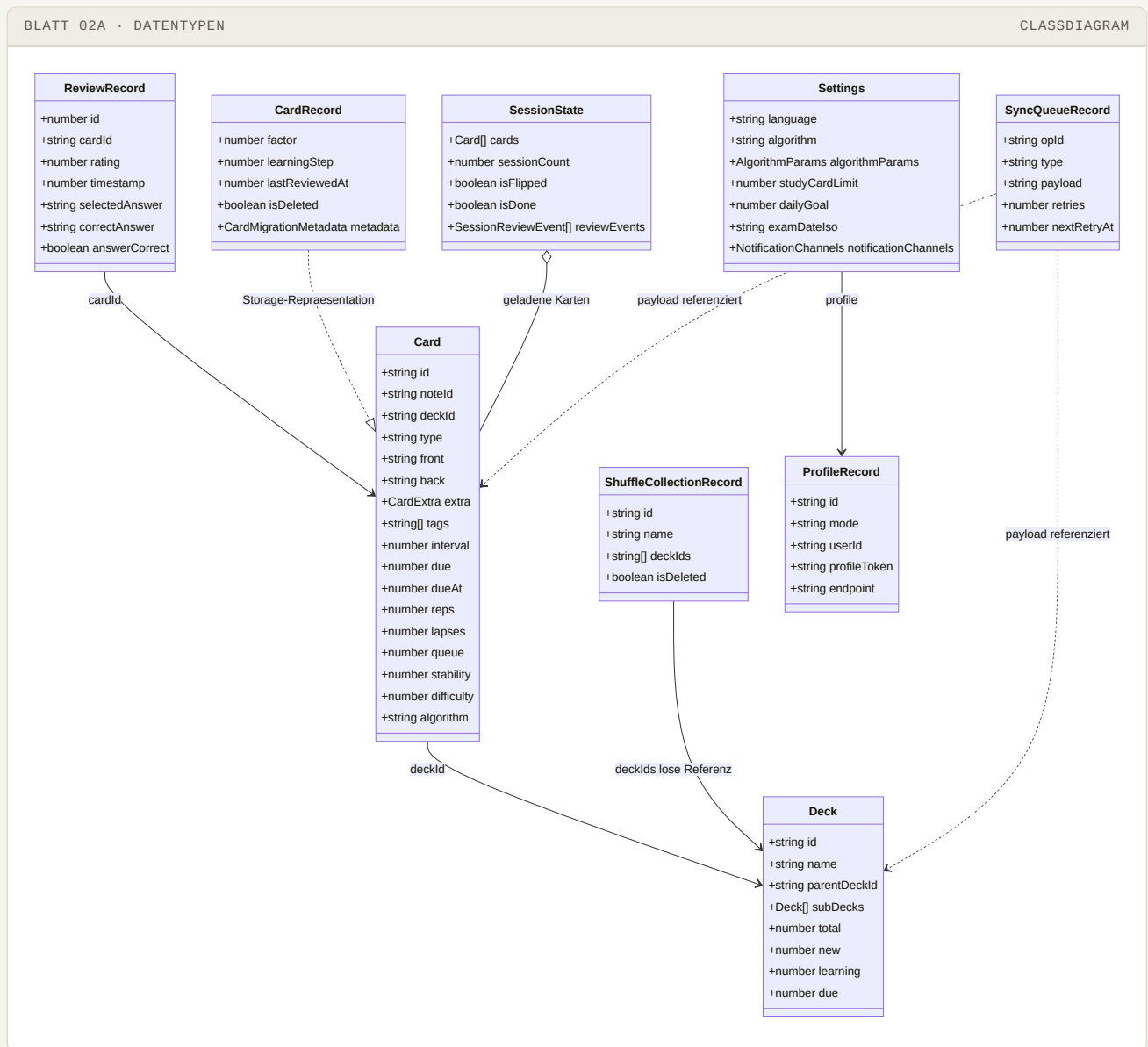
FLOWCHART TD



Hinweis Schichtregel: Die Pfeile stellen die überwiegende Richtung dar. Eine Ausnahme durchbricht die Schichtregel bewusst — siehe §13
Kritische Kopplungen: `utils/learningUnits.ts` importiert `services/studyCardOrdering.ts` (Utils → Services).

§2

Zwei Teile: (a) zentrale Datentypen mit Beziehungen, (b) Komponenten-Dispatch/Komposition für die Karten-Renderer und Modal-Familien.



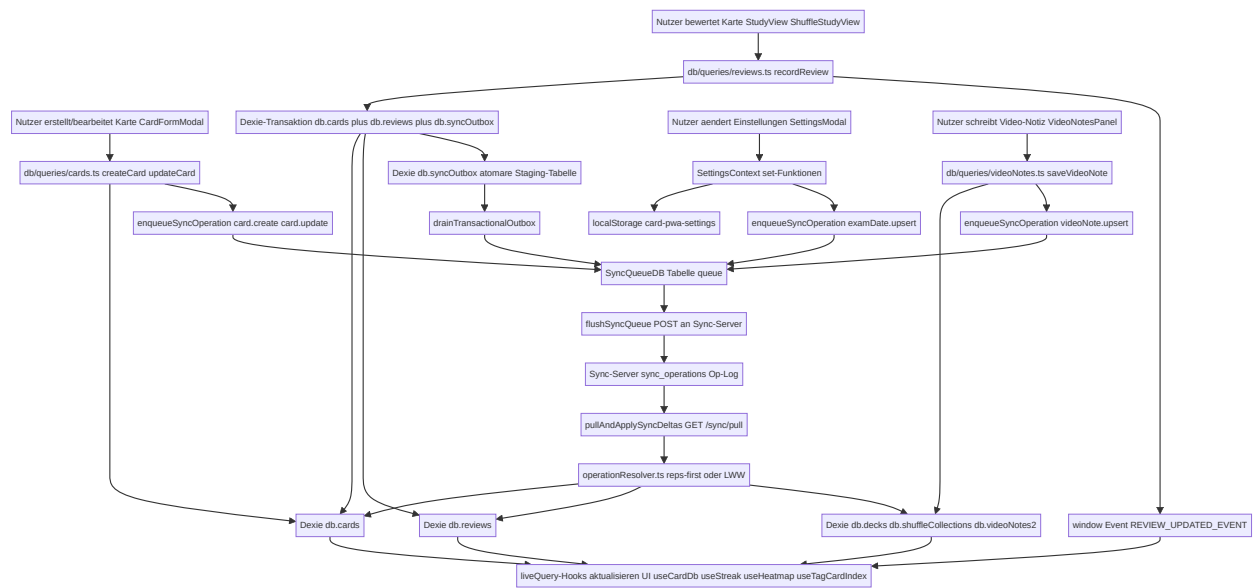
Nicht alle 74 Komponenten sind hier abgebildet — vollständige Zuordnung siehe §16 Abhängigkeitstabelle. Die Diagramme zeigen die architektonisch bedeutsamsten Vererbungs-/Kompositions-/Dispatch-Beziehungen.

S3 Datenflussdiagramm

Wie Karten-, Review-, Settings- und Video-Notiz-Daten erzeugt, verändert, weitergereicht und gespeichert werden — inklusive Sync-Push/Pull.

BLATT 03 · DATENFLUSS

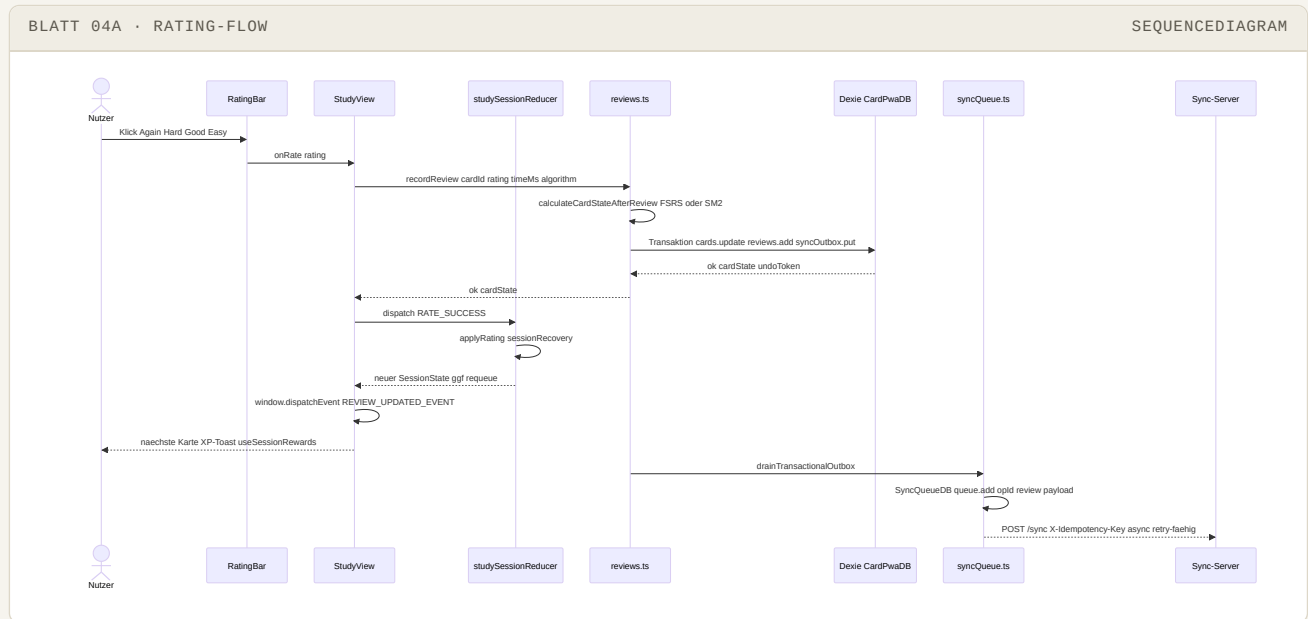
FLOWCHART TD



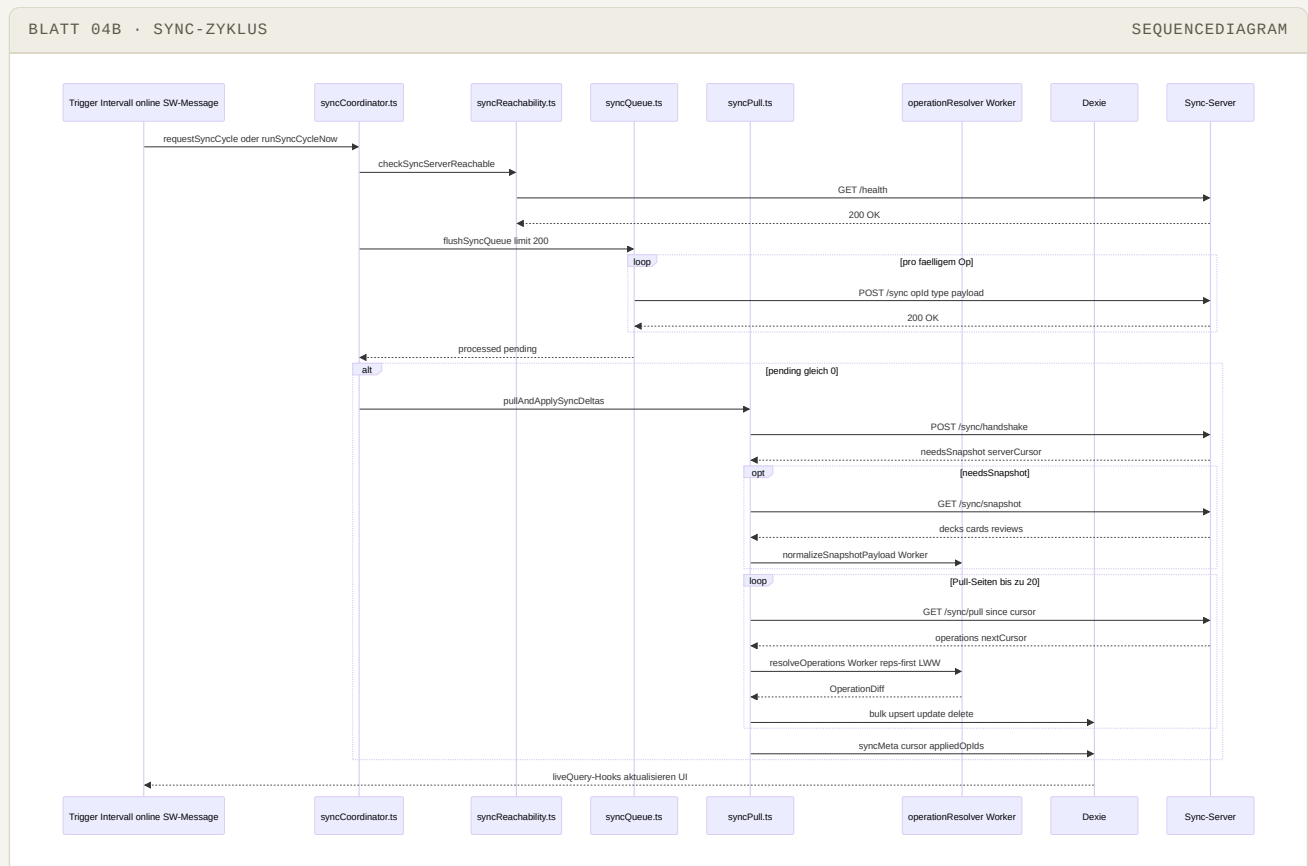
§4 Ablaufdiagramme zentraler Anwendungsfälle

Vier repräsentative End-to-End-Abläufe vom UI-Ereignis bis zur Speicherung bzw. UI-Aktualisierung.

4.1 Karte lernen & bewerten



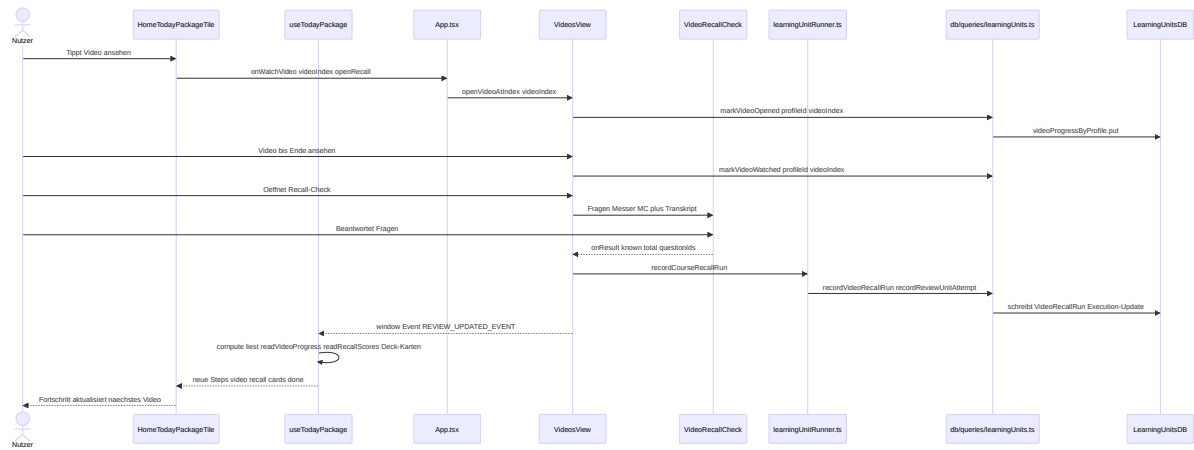
4.2 Sync-Zyklus (Push vor Pull)



4.3 Heute-Paket: Video ansehen und Recall-Check

BLATT 04C · HEUTE-PAKET / VIDEO-LERNEN

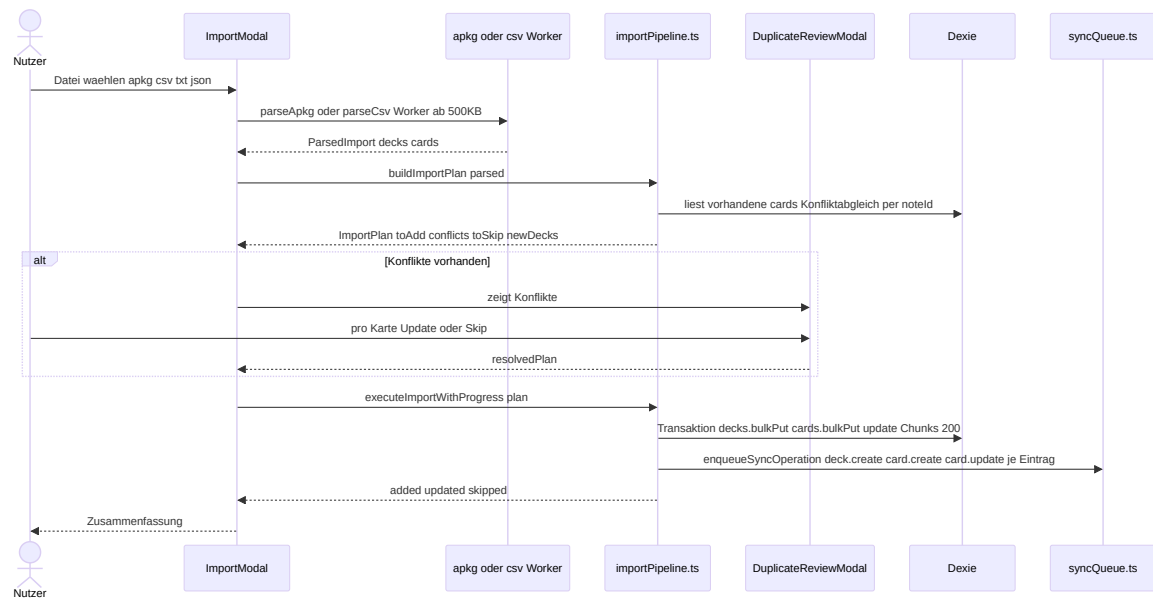
SEQUENCEDIAGRAM



4.4 Import (Anki APKG / CSV / JSON-Backup)

BLATT 04D · IMPORT-PIPELINE

SEQUENCEDIAGRAM



\$5 Einstiegspunkte der Anwendung

`index.html`

Lädt `src/main.tsx` als Modul; keine externen Render-Blocking-Stylesheets (Fonts selbst gehostet über `public/fonts.css`).

`src/main.tsx`

Bootstrap: `migrateCardPwaBrandingData()` (einmalige Rebrand-Migration alter Dexie-DBs/localStorage-Keys) → `initServiceWorkerRegistration()` → `ReactDOM.createRoot().render(<App startupReady>)`. Kein Router, keine weiteren Provider hier.

`src/App.tsx` (910 Zeilen)

Eigentlicher Anwendungs-Einstiegspunkt: `AppShell` hält den View-State, initialisiert Splash/Update-Overlays, verdrahtet `ThemeProvider/SettingsProvider/LazyMotion` und rendert lazy-geladene Views.

`public/service-worker.js` (1559 Zeilen)

Zweiter, unabhängiger Einstiegspunkt (eigener Thread): App-Shell-Precache, Offline-Fallback-Seite, `fetch/sync/periodicsync/push/message-Handler`. Handgeschrieben, nicht über vite-plugin-pwa generiert.

Deep-Links / Query-Parameter

`?view=study` (Schnellstart Tageslern-Session), `?view=shuffle/shuffle-manage`, `?view=import` (auch via Chromium File-Handling-API `window.launchQueue`), `?safeAreaDebug=1`.

S6 Verzeichnis- und Modulstruktur

VERZEICHNIS	DATEIEN	INHALT
src/	5	<code>App.tsx</code> , <code>main.tsx</code> , <code>env.ts</code> , <code>i18n.ts</code> (65 KB reine Übersetzungstabelle de/en), <code>vite-env.d.ts</code>
src/components/	44	Geteilte Views, Modals, Karten-Renderer, Bars/Panels/Sections
src/components/home/	16	Home-Dashboard-Bausteine, angesteuert vom Controller-Hook
src/components/videos/	7	Video-Player, Notizen, Recall-Check, Tag-System
src/components/labs/	3	PBQ-Lab-Szenarien (Matching/Ordering/Decision/Workflow)
src/components/acronyms/	2	Akronym-Quiz (in Bearbeitung, unversioniert)
src/hooks/	29	App-weite Hooks (Daten, Runtime, Video, PWA)
src/hooks/home/	8	Home- und Lerneinheiten-Controller-Hooks
src/services/	24	Sync-Protokoll, Study-Session-Engine, Domänenlogik
src/db/	3	<code>index.ts</code> (CardPwaDB), <code>learningUnitsDb.ts</code> (LearningUnitsDB), <code>queries.ts</code> (Re-Export-Kompatibilität)
src/db/queries/	14	Dexie-Zugriffsfunktionen, gruppiert nach Domäne
src/utis/	39	Algorithmen (FSRS/SM2), Parser, Scoring, Domänenlogik
src/utis/normalize/	6	Tolerante Normalisierung von Sync-/Backup-Payloads
src/utis/stats/	1	Reine Aggregationskerne (Heatmap/Streak/Forecast)
src/utis/sync/	1	Konfliktauflösung für Sync-Operationen
src/utis/workers/	7	Web-Worker-Wrapper + Pool-Fabrik
src/utis/import/	10	Anki/CSV/JSON-Import-Pipeline
src/data/	12	Statische/generierte Lerninhalte (SY0-701, Labs, Zitate)
src/contextts/	2	ThemeContext, SettingsContext
src/types/	1	App-facing UI/Domain-Typen (<code>index.ts</code> , 189 Zeilen)
src/constants/	3	Storage-Keys, SW-Kanäle, Events, UI-Design-Tokens
src/runtime/	1	Service-Worker-Registrierung (dependency-injected, testbar)
src/ui/	2	framer-motion-Shim (Bundle-Größen-Konvention)
src/vendor/	1	<code>fzstd.ts</code> — vendorierter Zstd-Dekompressor für Anki-Import
public/	1+	<code>service-worker.js</code> , Fonts, PWA-Icons, Manifest, Messer-Transkripte (JSON)

§7 Verantwortlichkeit jedes wichtigen Moduls

components/

Views (Vollbild, per view-State gerendert): `HomeView` ist bewusst „nur Klebstoff“ — Datenlogik liegt in `hooks/home/*`; `StudyView` / `ShuffleStudyView` sind die Study-Session-Engines (nahezu identischer Aufbau: `useReducer` (`sessionReducer`), Karten-Restore aus `IndexedDB`, Rating-Flow, Coach-Panel); `VideosView` (1286 Zeilen, größte Feature-Komponente) bündelt Player, Notizen, Recall-Check, Transkript, Tag-System und Kurs-Executions; `LearningUnitsView` ist die Vollbild-Ansicht des additiven SY0-701-Lerneinheiten-Systems; `LabsView` listet PBQ-Szenarien; `AcronymPracticeView` ist ein Self-Assessment-Quiz ohne Scheduling-Nebenwirkung.

Karten-Renderer: `CardFace` ist reiner Dispatcher (parst Fronttext, wählt Renderer); `OrderingCard` / `MatchingCard` / `DragMatchCard` / `FreeRecallCard` sind alle lazy-geladene, gleichwertige Interaktionstypen mit fast identischem Props-Vertrag.

Modals: `CardFormModal` ist der einzige echte Karten-Editor (Create/Edit sind dünne Adapter darüber); `SettingsModal` (1734 Zeilen, größte Einzelkomponente) bündelt fünf Domänen; `ImportModal` + `DuplicateReviewModal` bilden die Import-UI.

hooks/

Drei Kategorien: **Daten-Hooks** (`useCardDb.ts` mit `useDecks/useDeckCards/useStats/useGamificationProfile/useShuffleCollections/useShuffleCards`, `useVideoNotes.ts`, `useVideoTags.ts`) — alle auf Dexie `liveQuery` + gemeinsamem `Debounce` aufgebaut; **Controller-Hooks** (`hooks/home/useHomeController.ts` — zentraler UI-State-Hub für `HomeView` mit ~22 State-Feldern und ~30 Handlern; `useHomeDerivedData.ts`, `useLearningUnits.ts`, `useTodayPackage.ts` — asynchrone Ableitungen aus mehreren DB-Quellen); **Runtime-/Utility-Hooks** (`useSyncRuntime`, `useServiceWorkerConfig`, `useAppBadge`, `useWakeLock`, `useVisualViewport` usw. — meist in `AppInitializer.tsx` gebündelt).

services/

Sync-Protokoll (`syncConfig`, `syncQueue`, `syncPull`, `syncCoordinator`, `syncReachability`, `syncDeckScope`) — Push-Queue + Pull-Cursor Protokoll, siehe §4.2. **Study-Engine** (`studySessionReducer` — Redux-artiger Reducer, `studySessionPersistence`, `studyCardOrdering`, `studyModeSelector`, `sessionRecovery`, `cardOrdering`) — Auswahl/Reihenfolge/Wiedervorlage von Karten, unabhängig vom Scheduling-Algorithmus selbst. **Domänenservices:** `profileService` (Geräte-/Profil-Identität + REST-Client für `/auth/*`), `deckHierarchy`, `learningUnitRunner` (Orchestrierung des Lerneinheiten-Systems, schreibt nie Reviews/XP), `shuffleSession`, `learningCoach` (rein beratend, keine Persistenz), `fsrsOptimizer` (WASM-Parameteroptimierung), `algorithmMigration`, `brandMigration`, `webPush`, `offlineStatus`, `errorLog`.

db/

`db/index.ts` definiert `CardPwaDB` (Dexie, 21 Versionsschritte inkl. mehrerer Daten-Migrationen) mit 14 Tabellen. `db/learningUnitsDb.ts` ist eine bewusst getrennte zweite Dexie-Datenbank für das additive Lerneinheiten-System (10 Tabellen, alle Keys beginnen mit `profileId`). `db/queries/*` kapselt sämtliche Dexie-Zugriffe nach Domäne; nur `answerStats.ts`, `learningUnits.ts` und `sharedReads.ts` sind bewusst nicht über den Barrel-Export (`db/queries/index.ts`) erreichbar.

utils/

Scheduling-Algorithmen: `fsrs.ts` (ts-fsrs-Adapter) und `sm2.ts` (reines SM-2, austauschbar über identische Signatur), gemeinsam parametrisiert über `algorithmParams.ts`. **Parsing/Scoring:** `cardTextParser.ts` (zentrale Text → Frage/Antwort-Übersetzung für alle Kartentypen), `cardVariant.ts`, `pbqScoring.ts`, `dragMatchScoring.ts`, `freeRecallScoring.ts`. **Sync-Unterstützung:** `normalize/*` (tolerante Payload-Validierung), `sync/operationResolver.ts` (Konfliktlogik, Worker-fähig). **Web Worker:** `workers/*` — APKG-Parsing, CSV-Parsing, Snapshot-Normalisierung, Sync-Op-Auflösung, Statistik-Aggregation, jeweils mit synchronem Main-Thread-Fallback über `workerPool.ts`. **Import:** `import/*` — vollständige Anki-`(.apkg/.colpkg)`-, CSV/TXT- und JSON-Backup-Pipeline. **Domänenlogik:** `learningUnits.ts` (Kern des Lerneinheiten-Systems, größte Utils-Datei nach Exportfläche), `learningUnitRanking.ts`, `securityDeckHierarchy.ts` (statische SY0-701-Struktur), `todayPackage.ts`, `tagIdentity.ts` + `videoTags/videoLinks/videoTimeAnchors/videoTagStats/videoTagPageData` (Obsidian-artiges Tag-System für Video-Notizen).

data/ (statische/generierte Inhalte)

12 Dateien, größtenteils generiert (Header-Kommentar „GENERATED“): `sy0701Requirements.ts` (12.907 Zeilen, größte Datendatei — vollständiges Exam-Requirements-Manifest), `sy0701ContentMap.ts` (4512 Zeilen — Video ↔ Objective ↔ Karten-Zuordnung), `sy0701Acronyms.ts` (336 Akronym-Paare), `sy0701Coverage.ts`, `labScenarios.ts` (3703 Zeilen, kuratierte PBQ-Szenarien), `labBlueprints.ts` (Generator-Pools), `messengerTranscriptQuestions.ts` (7476 Zeilen), `messengerVideoQuestionMap.ts`, `professorMesserLinks.ts`, `motivationQuotes.ts`, `aiModeGuides.ts`, sowie das neue (unversionierte) `acronymDefinitions.ts`.

S8 Zentrale Datenmodelle

Es existieren bewusst zwei Ebenen: *Storage-Records* (definiert in `db/index.ts`) bzw. `db/learningUnitsDb.ts`) und *App-facing Typen* (definiert in `types/index.ts`), die per Query-Mapping-Funktionen (z. B. `mapCard`, `mapDeck` in `db/queries/decks.ts`) ineinander überführt werden.

Card / CardRecord

Vereinheitlichtes Feld-Set für SM-2 und FSRS: `type (new/learning/review/relearning)`, `queue`, `due` (Tages-Index, legacy), `dueAt` (Epoch-ms, aktuell), `learningStep`, `lastReviewedAt`, `interval`, `factor` (SM-2-Ease), `stability/difficulty` (FSRS), `reps`, `lapses`, `algorithm`, `Soft-Delete` (`isDeleted/deletedAt`), `metadata` (Snapshot für reversible Algorithmus-Migration), `extra` (Akronym/Beispiele/Port/Protokoll), `tags: string[]`.

Deck / DeckRecord

`id`, `name`, `parentDeckId` (Hierarchie), aggregierte `DeckStats` (`total/new/learning/due`), rekursiv verschachtelte `subDecks` (nur im App-facing Typ, zur Laufzeit aus flacher Tabelle gebaut).

ReviewRecord

Auto-Increment-id, `cardId`, `rating` (1–4), `timestamp`, optional `selectedAnswer/correctAnswer/answerCorrect` für interaktive Kartentypen.

ShuffleCollectionRecord

`id`, `name`, `deckIds: string[]` (lose Referenz — Karten werden nie kopiert), `Soft-Delete`.

ProfileRecord (PK 'current')

`mode: 'local' | 'linked'`, bei `linked`: `userId`, `profileToken`, `endpoint`.

Settings (SettingsContext, nicht Dexie)

~30 Felder: `Sprache`, `Algorithmus + algorithmParams`, `Studien-Limits` (`studyCardLimit`, `dailyGoal`, `newCardsPerDay`, `recallCheckSize`, `learnAheadMinutes`), `Hard-Practice-Reinforcement-Parameter`, `examDateIso/examDateUpdatedAt`, `notificationChannels` (5 Kanäle), `Theme/Font-Präferenzen`.

SessionState / PersistedStudySession

`Laufzeit-Reducer-State` (`services/studySessionReducer.ts`): `geladene cards`, `Flip/Done-Flags`, `Recovery-Zähler` (`againCounts`, `lowRatingCounts`, `hardPracticeCardIds/PassCounts`), `reviewEvents`. Persistierte Variante (Version 5, TTL 45 min) in `db.activeSessions`.

SyncQueueRecord / SyncOperationType

13 Operationstypen: `review`, `review.undo`, `card.create/update/delete`, `card.schedule.forceTomorrow`, `deck.create/delete`, `shuffleCollection.upsert/delete`, `videoNote.upsert/delete`, `progress.reset`, `examDate.upsert`.

VideoNoteRecord / VideoTagMetaRecord

`VideoNoteRecord` (PK [`profileId`, `objective`]): `Freitext-Inhalt`, `tags` werden aus inline `#tag-Syntax` im Text abgeleitet, nicht separat eingegeben. `VideoTagMetaRecord` (PK [`profileId`, `tagId`]): `Label/Beschreibung/Farbe/Pin/Aliase/Archiviert` — `Notiztext` bleibt Quelle der Wahrheit für Tag-Existenz.

Lerneinheiten-Familie (LearningUnitsDB)

`ProfileLearningStateRecord`, `LearningUnitState`, `LearningUnitExecution` (discriminated union `course/review/lab/exam`, „eingefroren“ beim Start gegen die Content-Map validiert), `ReviewUnitAttemptRecord` (append-only), `VideoProgressRecord`, `VideoRecallRun` (append-only), `DraftLearnerExamPlanRecord`, `LabAttemptRecord` (Szenario-Snapshot eingefroren bei Start).

§9 Zustandsverwaltung

Kein globaler Store (kein Redux/Zustand/Jotai). Fünf parallele Mechanismen, je nach Datenlebensdauer:

- **React Context** — `ThemeContext` (aktuell nur ein Theme 'default'; `setTheme`/Multi-Theme-Mechanik ist vorhandene, aber ungenutzte Flexibilität) und `SettingsContext` (App-weite Einstellungen + Profil, persistiert nach `localStorage` bzw. Dexie `profile`-Tabelle).
- **Lokaler Komponentenstate** — `App.tsx` hält den View-State und ~20 weitere `useState`-Felder direkt in `AppShell` (keine Auslagerung in einen Reducer, obwohl die Zustandsfläche groß ist).
- **useReducer** — `services/studySessionReducer.ts` ist ein echter Redux-artiger Reducer `((state, action) => state)` mit 9 Action-Typen, genutzt identisch in `StudyView` und `ShuffleStudyView`.
- **Dexie LiveQuery als reaktiver Store** — Hooks wie `useDecks`, `useStats`, `useGamificationProfile`, `useVideoNote` abonnieren IndexedDB direkt; Änderungen propagieren automatisch ins UI, zusätzlich gebündelt/entprellt (180–250 ms) über eine gemeinsame globale `Revisions-liveQuery` in `useCardDb.ts`, um N parallele Tabellen-Scans zu vermeiden.
- **Controller-Hooks als „Presenter“-State** — `useHomeController.ts` bündelt reinen UI-State (welches Modal offen ist, Formularfelder) getrennt von `useHomeDerivedData.ts` (asynchron abgeleitete Lesedaten) — bewusste Trennung von UI-State und Server/DB-State.
- **Modul-Singletons** — `services/offlineStatus.ts` (`OfflineStatusService`-Klasse mit Subscriber-Liste), `hooks/useToast.ts` (Toast-Queue außerhalb von React, `useSyncExternalStore`-artiges Muster).

§10 Kommunikation zwischen Komponenten

- **Props/Callback-Drilling über Controller-Hooks** — der dominante Kommunikationsweg: Ein Controller-Hook (z. B. `useHomeController`) liefert ein großes Objekt aus State + Handlern, das 1:1 an mehrere Kind-Komponenten weitergereicht wird (dokumentiert in der Feld → Komponente-Zuordnung, siehe Hooks-Recherche). Kein Context für Feature-lokalen State.
- **Globale window-Custom-Events** — zentrales Invalidierungssignal `REVIEW_UPDATED_EVENT` (`constants/appIdentity.ts`), gefeuert nach jeder Review-Schreiboperation, abonniert von praktisch allen Dexie-Lese-Hooks. Weitere: `EXAM_DATE_SYNCED_EVENT` (Sync → SettingsContext), `SW_CHANNELS.updateEvent` (Service-Worker → App.tsx), `SYNC_RUNTIME_CONFIG_CHANGED_EVENT` (Profil-/Token-Wechsel → Sync-Runtime-Neustart).
- **postMessage App ↔ Service Worker** — bidirektionales Protokoll mit ~15 Nachrichtentypen (`SKIP_WAITING`, `SYNC_CONFIG`, `DAILY_REMINDER_CONFIG`, `SW_NOTIFICATIONS_CONFIG`, `KPI_DUE_COUNT`, `SESSION_SNAPSHOT`, `REGISTER_SYNC`, `REGISTER_PERIODIC_SYNC`, `FORCE_SYNC_NOW`, `PREFETCH_URLS`, `SERVER_STATUS_NOTIFICATION`, `PUSH_SUBSCRIPTION_CHANGED` u. a.) — siehe `public/service-worker.js` Zeile 1331 ff. Nicht durch TypeScript über die Thread-Grenze hinweg geprüft.
- **Web-Worker-Nachrichten** — einheitliches Request/Response-Muster (`id/requestId` → `ok/result/error`) für APKG-/CSV-Parsing, Snapshot-Normalisierung, Sync-Op-Auflösung und Statistik-Aggregation, mit synchronem Fallback bei Worker-Fehlern.
- **Dexie LiveQuery als Pub/Sub** — De-facto-Kommunikationskanal zwischen Schreiboperationen (`db/queries/*`) und Lese-Hooks, ganz ohne explizite Event-Emission.

§11 Externe und interne Abhängigkeiten

Externe npm-Abhängigkeiten (Laufzeit-relevant)

PAKET	ZWECK	EINGESETZT IN
react	UI-Framework (React 18, createRoot)	überall
dexie	IndexedDB-Wrapper, liveQuery	db/*, alle Daten-Hooks
framer-motion	Animation (m/AnimatePresence via Shim, Reorder direkt)	ui/motion.ts, LabScenarioView, DragMatchCard
ts-fsrs	FSRS-Scheduling-Kern	utils/fsrs.ts
@open-spaced-repetition/binding	FSRS-Parameteroptimierung (WASM/WASI, Worker)	services/fsrsOptimizer.ts
@dnd-kit/core, @dnd-kit/sortable	Drag-and-Drop (PBQ Ordering)	components/OrderingCard.tsx
jszip	Entpacken von .apkg/.colpkg	utils/import/apkgParserCore.ts
sql.js	SQLite-Parsing im Browser (WASM)	utils/import/ankiDatabase.ts
papaparse	CSV-Parsing	utils/import/csvImporter.ts
ogl	WebGL2 (Metaballs-Deko-Animation)	components/MetaBalls.tsx
lucide-react (o.ä.)	Icon-Set	diverse Komponenten
fzstd (vendoriert, nicht npm)	Zstd-Dekompression für collection.anki21b	src/vendor/fzstd.ts → ankiDatabase.ts

Interne Schichtregel

Grundsätzlich: *Components* → *Hooks* → *Services* → *DB/Queries* → *Utils*, mit Utils als „unterste“, größtenteils abhängigkeitsfreie Schicht. Diese Regel wird an einer Stelle durchbrochen (siehe §13): `utils/learningUnits.ts` importiert `buildTodayPackageSelection` aus `services/studyCardOrdering.ts` — eine Utils-Datei hängt damit von der Services-Schicht ab.

§12 API- und Speicherzugriffe

REST-Endpunkte (Sync-Server)

Basis: syncBaseEndpoint (endet auf /sync); authBase = derselbe String ohne /sync-Suffix. Auth per Authorization: Bearer <profileToken>.

METHODE	PFAD	ZWECK	MODUL
POST	authBase/auth/profile	Server-Profil anlegen	profileService.ts
POST	authBase/auth/pair/issue	Pairing-Code ausstellen (Bearer)	profileService.ts
POST	authBase/auth/pair/redeem	Pairing-Code auf Zweitgerät einlösen	profileService.ts
POST	authBase/auth/recover	Profil per Recovery-Code wiederherstellen	profileService.ts
POST	authBase/auth/revoke	Geräte-Token widerrufen (Bearer)	profileService.ts
POST	authBase/auth/device/remove	Gerät serverseitig entfernen (Bearer)	profileService.ts
GET	authBase/auth/profiles?limit=N	Profile für Wechsel-UI auflisten	profileService.ts
POST	authBase/auth/profile/switch	Gerät an anderes Profil binden	profileService.ts
GET	authBase/auth/public-profiles	Öffentliche/Familien-Profil + Deck-Anzahl	profileService.ts
POST	authBase/auth/profile/join	Öffentlichem/Default-Profil beitreten, optional joinPin	profileService.ts
GET	authBase/auth/default-profile	Default-Profil abrufen (ohne Auth)	profileService.ts
GET	authBase/sync/decks	Verfügbare Decks für verknüpftes Profil	profileService.ts
POST	syncBase	Einzelne Sync-Operation pushen (Idempotency-Key)	syncQueue.ts
POST	syncBase/handshake	Bootstrap-Verhandlung (Counts/Cursor)	syncPull.ts
GET	syncBase/snapshot?clientId=	Vollständigen Zustand herunterladen	syncPull.ts
POST	syncBase/bootstrap/upload	Vollständigen Zustand hochladen (neues Gerät)	syncPull.ts
GET	syncBase/pull?since=&limit=&clientId=	Paginierter Delta-Pull des Op-Logs	syncPull.ts
GET	authBase/health	Erreichbarkeits-Check	syncReachability.ts
POST	.../push/subscribe	Web-Push-Abo registrieren	webPush.ts
GET	/media/messer/index.json	Video-Manifest (Professor-Messer-Kurs)	hooks/useLocalMesserVideos.ts, utils/todayPackage.ts
GET	/messer-transcripts/NNN.json	Video-Transkript	hooks/useMesserVideoTranscript.ts

IndexedDB (3 getrennte Dexie-Datenbanken)

DATENBANK	TABELLEN	ZWECK
CardPwaDB (card-pwa-db, v1-v21)	decks, cards, reviews, activeSessions, syncMeta, syncOutbox, profile, cardStats, deckProgress, shuffleCollections, videoNotes2, videoDownloads, videoBlobs, videoTagMeta	Haupt-Anwendungsdaten
LearningUnitsDB (card-pwa-learning-units, v1-v2)	profileLearningState, learningUnitState, unitExecutions, reviewUnitAttempts, videoProgressByProfile, videoRecallRuns, legacyAssessmentHints, learnerExamPlans, migrationMeta, labAttempts	Additives SY0-701-Lerneinheiten-System, komplett getrennt von CardPwaDB
SyncQueueDB (card-pwa-sync-queue, v1)	queue	Durable Retry-Queue für ausgehende Sync-Operationen

localStorage (Auswahl zentraler Keys, siehe constants/appIdentity.ts → STORAGE_KEYS)

KEY	MODUL	INHALT
card-pwa-settings	contexts/SettingsContext.tsx	Vollständiges Settings-Objekt als JSON
theme	contexts/ThemeContext.tsx	Gewählter Theme-Key
Video-Fortschritt/Recall-Scores	hooks/useMesserVideoProgress.ts, useVideoRecallScores.ts	Watched/Confidence-Map bzw. Recall-Run-Historie je Video
Home-Dashboard-Zustand	hooks/home/homeControllerHelpers.ts, useHomeDeckFilters.ts	Dashboard-Modus, Sortierung, Shuffle-Only-Toggle
Today-Package-Pointer/Video-Katalog	utils/todayPackage.ts	Zeiger auf aktuelles Kurs-Video, Offline-Cache der Datei-Liste
algorithmMigration / algorithmDiagnostics	services/algorithmMigration.ts, db/queries/diagnostics.ts	Migrations-Log bzw. Dev-Diagnose-Log
errorLog	services/errorLog.ts	Ringpuffer der letzten 500 Client-Fehler
card-pwa-device-id, card-pwa-sync-client-id, Profil-Deck-Auswahl je Nutzer	services/profileService.ts, syncConfig.ts	Geräte-/Client-Identität, pro-Profil Sync-Scope
Legacy-Sync-Cursor (Fallback)	services/syncPull.ts	Vorgänger von db.syncMeta — Beispiel für laufende localStorage → Dexie-Migration im Code

sessionStorage: pendingImportRequest (überlebt SW-getriggerten Reload beim Datei-Import), Offline-Retry-Zähler in der SW-Offline-Fallback-Seite.

§13 Kritische Kopplungen und zyklische Abhängigkeiten

- **SCHICHT-INVERSION** `utils/learningUnits.ts` → `services/studyCardOrdering.ts`: Die sonst abhängigkeitsfreie Utils-Schicht importiert für `selectCourseCardIds` aktiv aus der Services-Schicht (`buildTodayPackageSelection`). Da `services/learningUnitRunner.ts` umgekehrt `utils/learningUnits.ts` importiert, entsteht ein architektonischer Ring über zwei Schichten (kein echter Modul-Zyklus, aber eine Umkehrung der dokumentierten Schichtregel).
- **LOGIK-DUPLIKATION** **Deck-Sync-Scope dreifach implementiert**: `services/syncedDeckScope.ts` (`getSyncedDeckIds`), `services/syncQueue.ts` (`getSelectedDeckFilter/shouldSyncOperation`) und `services/syncPull.ts` (`readSelectedDeckFilter/shouldApplyOperationForSelectedDecks`) implementieren dieselbe Semantik unabhängig voneinander — Drift-Risiko bei künftigen Änderungen an der Scope-Logik.
- **REDUNDANTE REIHENFOLGE-GARANTIE** Push-vor-Pull wird *zweimal* durchgesetzt: einmal in `syncCoordinator.ts` (Phasenreihenfolge) und ein zweites Mal innerhalb von `syncPull.ts` s eigenem `pullAndApplySyncDeltas` (interner Pending-Check vor dem Pull).
- **DATEI-ZU-DATEI-KOPPLUNG** `components/home/HomeLearningUnitsTile.tsx` importiert das Wörterbuch `LEARNING_UNIT_COPY` direkt aus der Nachbardatei `HomeLearningUnitList.tsx`, obwohl die Tile die Liste nicht rendert.
- **TYP-DUPLIKATION** Das lokale Union `HomeTab` (`'dashboard' | 'decks' | 'tags' | 'learning-units' | 'daily-quest' | 'labs' | 'acronyms'`) ist unabhängig in `HomeBottomBar.tsx` und `HomeDeckToolbar.tsx` dupliziert statt aus einer gemeinsamen Quelle importiert.
- **NAMING-INKONSISTENZ** `db/learningUnitsDb.ts` verwendet den Literal-String `'card-pwa-learning-units'` statt eines Eintrags in `constants/appIdentity.ts` → `DATABASE_NAMES` (das nur `app/legacyApp/syncQueue/legacySyncQueue` kennt) — Rename-Risiko.
- **DOPPELTE AKKORDEON-IMPLEMENTIERUNG** `SettingsSection.tsx` und die lokale `FaqSection` in `FaqModal.tsx` sind funktional nahezu identische, unabhängig implementierte `Accordion-Wrapper`.
- **MODAL-SHELL-INKONSISTENZ** Von ~10 Modals komponieren nur `InstallHintModal` und `FutureForecastModal` tatsächlich `ModalShell.tsx`; die übrigen (u. a. `SettingsModal`, `CardFormModal`, `ImportModal`) bauen ihr `Overlay-Markup` direkt über `UI_TOKENS.modal.*` nach.

§14 Nicht verwendete oder möglicherweise veraltete Funktionen

Per Grep über `src/` (ohne Tests) ermittelt. Als „unklar“ markierte Punkte sind nicht abschließend beweisbar aus statischer Analyse allein.

- **UNGENUTZT** `services/updateService.ts` (`checkForAppUpdates`) — kein Verwender außerhalb von Tests gefunden; die tatsächliche SW-Update-Erkennung läuft komplett über `runtime/swRegistration.ts` + die eigenen `useEffect`-Listener in `App.tsx`. Wirkt wie eine abgelöste, aber nicht entfernte Alternative.
- **UNGENUTZT** `useVideoTagMeta` und `useResolvedVideoTag` aus `hooks/useVideoTags.ts` — kein Komponenten-Verwender gefunden.
- **NUR INTERN VERWENDET** `useObjectivesWithNotes` aus `hooks/useVideoNotes.ts` — nur von `useVideoNoteIndex` im selben Modul genutzt, nicht direkt von Komponenten.
- **NICHT VERDRAHTET** `utils/labGenerator.ts` + `utils/labTraining.ts` (deterministischer „unendlicher“ Lab-Varianten-Generator, ≥ 9999 Varianten laut Test) — kein Laufzeit-Verwender außerhalb von Tests gefunden. `LabsView.tsx` nutzt stattdessen ausschließlich die kuratierten `LAB_SCENARIOS` aus `data/labScenarios.ts`. Wirkt wie ein gebautes, aber nie an die UI angebundenes Feature.
- **NUR VON EIGENEM TEST IMPORTIERT** `data/aiModeGuides.ts` — Forschungs-/Logik-Leitfaden pro Lernmodus, wird aktuell nur von `__tests__/utils/ai-mode-guides.test.ts` referenziert, nicht von Laufzeit-Code.
- **VESTIGIALE ABSTRAKTION** `env.ts` (`RuntimeTarget = 'web'` als einziger Wert) und `Window.__TAURI__`-Ambient-Typen in `vite-env.d.ts` — Reste einer offenbar entfernten Multi-Target-/Tauri-Desktop-Option.
- **VESTIGIALE FLEXIBILITÄT** `contexts/ThemeContext.tsx`: `THEMES` hat nur den Schlüssel `'default'`; `setTheme` und `normalizeSavedThemeKey` (migriert jeden alten Wert stets zu `'default'`) sind Multi-Theme-Infrastruktur ohne aktuell zweites Theme.
- **TOTE TABELLE (HISTORISCH)** Die Dexie-Tabelle `videoNotes` (Singular) wurde in `db/index.ts` Version 19 entfernt (Daten nach `videoNotes2` migriert) — im aktuellen Schema nicht mehr referenzierbar, nur noch in der Versions-Historie sichtbar.
- **FALSCH-POSITIV KORRIGIERT** `db/learningUnitsDb.ts` trägt den Kopfkomentar „nothing in the existing app imports this yet“ — das ist inzwischen **veraltete Dokumentation, kein toter Code**: die Datenbank wird aktiv von `useLearningUnits.ts`, `learningUnitRunner.ts`, `LearningUnitsView.tsx`, `VideosView.tsx`, `SettingsContext.tsx`, `syncPull.ts` und `dbBackup.ts` verwendet.
- **UNKLAR** `components/MetaBalls.tsx` (WebGL2-Deko-Animation) — in der Recherche der übrigen Kern-Komponenten kein Einbindungsort gefunden; nicht eindeutig bestimmbar, ob sie aktuell eingebunden ist oder ein vorbereitetes/entferntes Splash-Element ist.

Stellen mit hoher Komplexität oder schwer nachvollziehbarem Datenfluss

- **App.tsx** (910 Zeilen) — manuelle View-State-Machine mit ~20 useState-Feldern und 11 useEffect-Blöcken in einer einzigen AppShell-Funktion, darunter mehrstufige, zeitbasierte Timeout-Kaskaden für das Zusammenspiel aus Splash-Screen, Service-Worker-Update-Overlay und dem Guard „nicht während einer aktiven Lernsession reloaden“ (isStudyView + pendingReloadAfterStudy). Der zeitliche Zusammenhang der Effekte ist nur durch Lesen aller elf Blöcke gemeinsam nachvollziehbar.
- **services/syncPull.ts** (1399 Zeilen) — Handshake/Snapshot/Bootstrap-Upload/Delta-Pull/Apply in einer Datei, mit bedingtem Worker-Offload, Legacy-localStorage-Cursor-Fallback und pro-Operationstyp unterschiedlicher Konfliktstrategie (reps-first vs. reines LWW vs. Force-Delete bei server-maintenance-publisher) verteilt über einen großen switch.
- **components/SettingsModal.tsx** (1734 Zeilen, größte Einzeldatei) — fünf fachlich unabhängige Domänen (Profil, Appearance, Learning, Notifications, Data/Danger-Zone) inklusive destruktiver „PWA Full Reset“-Aktion (Service-Worker-Deregistrierung, Cache-Löschung, IndexedDB-Löschung über vier Datenbanken, Cookie-/Storage-Bereinigung) in einer Komponente.
- **components/videos/VideosView.tsx** (1286 Zeilen) — größte Feature-Komponente; ein einzelnes State-Objekt koordiniert Player, Notizen, Recall-Check, Transkript, Tag-Sidebar, Tag-Collection-Panel und Kurs-Execution-Zustand gemeinsam.
- **Zusammenspiel** **studySessionReducer.ts** + **sessionRecovery.ts** — die Hard-Practice-Reinforcement-Schleife (erneutes Einreihen von Karten nach falscher Bewertung, begrenzt durch hardPracticeGoodStreak/hardPracticeMaxPasses) ist nur durch gemeinsames Lesen beider Dateien nachvollziehbar, da der Reducer die Requeue-Entscheidung an applyRating() aus der zweiten Datei delegiert.
- **Fünf-Datei-Verbund Lerneinheiten-System** — **LearningUnitsView.tsx** + **hooks/home/useLearningUnits.ts** + **utils/learningUnits.ts** + **utils/learningUnitRanking.ts** + **services/learningUnitRunner.ts** bilden gemeinsam ein additives Zusatzsystem mit eigenem Ranking-Algorithmus (Phasen/Pacing/Workload), eigenem Legacy-Import-Mechanismus und eigener Dexie-Datenbank — hohe Einstiegshürde für Änderungen, da fachliche Regeln über alle fünf Dateien verteilt sind.
- **components/CardFormModal.tsx** (749 Zeilen) — einziger Editor für vier strukturell unterschiedliche Kartenvarianten (Standard/MC/Ordering/Matching) mit dynamischem Template-Umschalten und eigener Text-Kodierungslogik pro Variante.

S16 Abhängigkeitstabelle

Gegliedert nach Schicht. Spalten wie gefordert: Element · Typ · Definiert in · Verwendet von · Abhängigkeiten · Verarbeitet Datenobjekte · Beschreibung. „Verwendet von“ nennt die wichtigsten Verwender, nicht zwingend erschöpfend bei sehr breit genutzten Utils.

16.1 Shell, Contexts, Views

ELEMENT	TYP	DEFINIERT IN	VERWENDET VON	ABHÄNGIGKEITEN	DATENOBJEKTE	BESCHREIBUNG
App	Komponente	App.tsx	main.tsx	ThemeContext, SettingsContext, AppInitializer, db/queries, alle Views (lazy)	View, Deck, Card, ShuffleCollection	View-State-Machine, Splash/Update-Overlay, Safe-Area-CSS-Vars
AppInitializer	Komponente	components/AppInitializer.tsx	App.tsx	9 Runtime-Hooks, services/deckHierarchy	—	Bündelt alle „fire-and-forget“-Init-Hooks (Sync, SW, Badge, Migration)
AppErrorBoundary	Komponente (Klasse)	components/AppErrorBoundary.tsx	App.tsx	services/errorLog	—	Einziger globaler Error-Boundary
ThemeContext / useTheme	Context	contexts/ThemeContext.tsx	App.tsx, viele Komponenten	constants/applidentity	Theme	Theme + CSS-Custom-Properties, getThemeChromeColor
SettingsContext / useSettings	Context	contexts/SettingsContext.tsx	fast alle Views/Modals	profileService, syncConfig, syncQueue, db/queries/learningUnits, utils/examDate, utils/algorithmParams	Settings, ProfileRecord	App-weite Einstellungen + Profil-Hydration
HomeView	View	components/HomeView.tsx	App.tsx	hooks/home/*, hooks/useCardDb, home/*-Komponenten	Deck, Card, ShuffleCollection, GamificationProfile	Dashboard-Shell, „Klebstoff“ ohne eigene Datenlogik
StudyView	View	components/StudyView.tsx	App.tsx	studySessionReducer, studyCardOrdering, studyModeSelector, db/queries/reviews, CardFace	Card, SessionState, ReviewRecord	Standard-Lernsession für ein Deck
ShuffleStudyView	View	components/ShuffleStudyView.tsx	App.tsx	wie StudyView + shuffleSession	Card, ShuffleCollection, SessionState	Lernsession über mehrere Decks hinweg
VideosView	View	components/videos/VideosView.tsx	App.tsx	learningUnitRunner, db/queries/learningUnits, videos/*-Komponenten	LocalVideoMeta, VideoNoteRecord, VideoRecallRun	Video-Workspace (Player+Notizen+Recall+Tags)
LearningUnitsView	View	components/LearningUnitsView.tsx	HomeView.tsx	useLearningUnits, learningUnitRunner, data/sy0701Requirements	LearningUnitExecution, DraftLearnerExamPlanRecord	SY0-701-Lerneinheiten-Vollansicht
LabsView	View	components/labs/LabsView.tsx	HomeView.tsx	data/labScenarios, learningUnitRunner, utils/labProgress	LabScenario, LabAttemptRecord	PBQ-Lab-Übersicht
AcronymPracticeView	View	components/acronyms/AcronymPracticeView.tsx	HomeView.tsx	utils/acronymQuiz, data/sy0701Acronyms, data/acronymDefinitions	AcronymMeaning	Self-Assessment-Quiz, kein Scheduling

16.2 Geteilte UI-Komponenten (Karten-Renderer, Modals, Bars/Panels)

ELEMENT	TYP	DEFINIERT IN	VERWENDET VON	ABHÄNGIGKEITEN	DATENOBJEKTE	BESCHREIBUNG
CardFace	Komponente	components/CardFace.tsx	StudyView, ShuffleStudyView	cardTextParser, cardVariant, utils/hash	Card	Dispatcher für Kartentypen Standard/Matching/Free Recall, Renderer
OrderingCard	Komponente (lazy)	components/OrderingCard.tsx	CardFace	@dnd-kit, utils/pbqScoring, utils/hash	Card	PBQ Reihenfolge-Übung
MatchingCard	Komponente (lazy)	components/MatchingCard.tsx	CardFace	utils/pbqScoring, utils/hash	Card	PBQ Zuordnung-Übung
DragMatchCard	Komponente (lazy)	components/DragMatchCard.tsx	CardFace	framer-motion, utils/dragMatchScoring, utils/hash	Card	M2 Drag-Matching-Optionen, 1
FreeRecallCard	Komponente (lazy)	components/FreeRecallCard.tsx	CardFace	utils/freeRecallScoring, utils/cardVariant	Card	M3 Freier Antwort-Selbsttest

ELEMENT	TYP	DEFINIERT IN	VERWENDET VON	ABHÄNGIGKEITEN	DATENOBJEKTE	BESCHRI
CardFormModal	Modal	components/CardFormModal.tsx	CreateCardModal, EditCardModal	db/queries/cards, utils/cardTextParser, utils/cardVariant, utils/sm2, utils/d	Card, DeckRecord	Kanonische Editor (alle ')
CreateCardModal / EditCardModal	Modal (Adapter)	components/CreateCardModal.tsx / EditCardModal.tsx	StudyView, HomeView, HomeDeckCardsModal	CardFormModal	Card	Dünne mod über CardFi
SettingsModal	Modal	components/SettingsModal.tsx	HomeView.tsx	~15 Services/Queries (siehe §15)	Settings, ProfileRecord, ErrorLogEntry	Gesamte Einstellungs Oberfläche
ProfileSyncSection	Section	components/ProfileSyncSection.tsx	SettingsModal.tsx	profileService, syncPull, syncQueue, syncCoordinator, syncReachability	ProfileRecord, ServerDeckSummary	Profil-Erstel Join/-Wechs Geräteverw
ImportModal	Modal	components/ImportModal.tsx	HomeView, SettingsModal	utils/import/*, utils/dbBackup	ParsedImport, ImportPlan	Anki/CSV/J Import-UI
DuplicateReviewModal	Modal	components/DuplicateReviewModal.tsx	ImportModal.tsx	utils/import/types	ImportPlan, ImportConflict	Konfliktaufl beim Import
DeckCard	Komponente	components/DeckCard.tsx	HomeDeckListSection	hooks/useFloatingMenu, DeckSchedulePanel, DeckTitleMarquee	Deck, DeckScheduleOverview	Deck-Kache rekursiv für
RatingBar	Bar	components/RatingBar.tsx	StudyView, ShuffleStudyView	—	Rating	4-Tasten-Be inkl. Haptik
StudyHeaderProgress	Komponente	components/StudyHeaderProgress.tsx	StudyView, ShuffleStudyView	ProgressBar, useSessionRewards	RewardHint	Fortschritt + XP/Combo-
SessionCoachPanel	Panel	components/SessionCoachPanel.tsx	StudyView, ShuffleStudyView	services/learningCoach	LearningCoachSummary	Beratendes Session-Fe
ReviewHeatmap	Komponente	components/ReviewHeatmap.tsx	home/HomeStatsSection.tsx	hooks/useHeatmap, ThemeContext	HeatmapBucket	Jahres-Akti Heatmap
ConfirmModal	Modal	components/ConfirmModal.tsx	HomeView, ProfileSyncSection, SettingsModal	—	—	Generische Dialog
ModalShell	Komponente	components/ModalShell.tsx	InstallHintModal, FutureForecastModal	—	—	Gemeinsam Modal-Chro von ~10 Mo
OfflineStatusBanner	Bar	components/OfflineStatusBanner.tsx	AppInitializer.tsx (immer gemountet)	services/offlineStatus, services/syncQueue	—	Netzwerk-/S Backlog-Hir
ToastContainer	Komponente	components/ToastContainer.tsx	App.tsx	hooks/useToast	—	Globale Toa Ausgabe

16.3 Feature-Komponenten (home/*, videos/*, labs/*)

ELEMENT	TYP	DEFINIERT IN	VERWENDET VON	ABHÄNGIGKEITEN
HomeHeaderBar	Bar	components/home/HomeHeaderBar.tsx	HomeView.tsx	StreakBadge, DailyGoalRing
HomeDeckListSection	Section	components/home/HomeDeckListSection.tsx	HomeView.tsx	DeckCard
HomeDeckToolbar / HomeBottomBar	Bar	components/home/HomeDeckToolbar.tsx / HomeBottomBar.tsx	HomeView.tsx	hooks/useFloatingMenu, MobileBottomSheet
HomeShuffleSection	Section	components/home/HomeShuffleSection.tsx	HomeView.tsx	—
HomeShuffleCollectionModal	Modal	components/home/HomeShuffleCollectionModal.tsx	HomeView.tsx	db/queries/shuffleCollections, services/shuffleSession
HomeStatsSection	Section	components/home/HomeStatsSection.tsx	HomeView.tsx	ReviewHeatmap, HomeDailyQuestTile, HomeQuestsPanel
HomeTagBrowseSection	Section	components/home/HomeTagBrowseSection.tsx	HomeView.tsx	constants/animations
HomeTodayPackageTile	Tile	components/home/HomeTodayPackageTile.tsx	HomeView.tsx	hooks/home/useTodayPackage
HomeLearningUnitList / HomeLearningUnitsTile	Section / Tile	components/home/HomeLearningUnitList.tsx / HomeLearningUnitsTile.tsx	HomeView.tsx, LearningUnitsView.tsx	utils/learningUnits, utils/learningUnitRanking
HomeExportModal / HomeCreateDeckModal / HomeDeckCardsModal	Modal	components/home/*.tsx	HomeView.tsx	hooks/useCardDb, utils/cardTextParser

ELEMENT	TYP	DEFINIERT IN	VERWENDET VON	ABHÄNGIGKEITEN
MesserVideoPlayer	Komponente	components/videos/MesserVideoPlayer.tsx	VideosView.tsx	hooks/useFullscreen, utils/videoPlayback
VideoNotesPanel	Panel	components/videos/VideoNotesPanel.tsx	VideosView.tsx	hooks/useVideoNotes, utils/tagSuggestions, utils/videoTags/Links/TimeAnchors, utils/videoNoteSignals
VideoRecallCheck	Modal	components/videos/VideoRecallCheck.tsx	VideosView.tsx, TagCollectionPanel.tsx	db/queries, data/messerTranscriptQuestions, data/messerVideoQuestionMap
VideoTranscriptPanel	Panel	components/videos/VideoTranscriptPanel.tsx	VideosView.tsx	hooks/useMesserVideoTranscript
TagCollectionPanel	Panel	components/videos/TagCollectionPanel.tsx	VideosView.tsx	hooks/useVideoNotes, useVideoTags, useMesserVideoProgress, db/queries
VideoTagSidebar	Sidebar	components/videos/VideoTagSidebar.tsx	VideosView.tsx	hooks/useVideoTags, utils/tagIdentity/videoTagStats
LabScenarioView	View (embedded)	components/labs/LabScenarioView.tsx	LabsView.tsx	utils/pbqScoring, labUi
AcronymDetailPanel	Panel (neu, unversioniert)	components/acronyms/AcronymDetailPanel.tsx	AcronymPracticeView.tsx	—

16.4 Hooks

ELEMENT	TYP	DEFINIERT IN	VERWENDET VON	ABHÄNGIGKEITEN	DATENOBJEKTE
useDecks / useDeckCards / useShuffleCollections / useShuffleCards	Hook	hooks/useCardDb.ts	HomeView, StudyView, ShuffleStudyView, HomeDeckCardsModal	db/queries/decks, shuffleCollections, dexie liveQuery	Deck, Card, ShuffleCollection
useStats / useGamificationProfile	Hook	hooks/useCardDb.ts	HomeView	db/queries/reviews, gamification, useDayStartMs	GlobalStats, GamificationProfile
useHomeController	Hook (Controller)	hooks/home/useHomeController.ts	HomeView.tsx	db/queries (createDeck, deleteDeck, deleteShuffleCollection), utils/dbBackup, services/webPush, homeControllerHelpers	Deck, ShuffleCollection
useHomeDerivedData	Hook	hooks/home/useHomeDerivedData.ts	HomeView.tsx	db/queries, services/shuffleSession, syncedDeckScope, utils/dbBackup, securityDeckHierarchy	DeckScheduleOverview, FutureForecastTile
useHomeDeckFilters	Hook	hooks/home/useHomeDeckFilters.ts	HomeView.tsx	utils/cardTextParser	Deck
useTagCardIndex	Hook	hooks/home/useTagCardIndex.ts	HomeView.tsx	db/queries/listAllCards, utils/tagIdentity, dexie liveQuery	Card
useTodayPackage	Hook	hooks/home/useTodayPackage.ts	HomeView, HomeTodayPackageTile, LearningUnitsView	db/queries, studyCardOrdering, utils/todayPackage, localVideoManifest	LocalVideoMeta,
useLearningUnits	Hook	hooks/home/useLearningUnits.ts	LearningUnitsView.tsx	utils/learningUnits, learningUnitRanking, db/queries/learningUnits, answerStats, learningUnitRunner	LearningUnitExecution, RankedLearningUnit
useCardDb / useVideoNotes-Familie / useVideoTags-Familie	Hook	hooks/useVideoNotes.ts / useVideoTags.ts	VideoNotesPanel, TagCollectionPanel, VideoTagSidebar, VideosView	db/queries/videoNotes, videoTagMeta, dexie liveQuery	VideoNoteRecord, VideoTagMetaRecord
useSessionRewards	Hook	hooks/useSessionRewards.ts	StudyView, ShuffleStudyView	db/queries/gamification, utils/gamification	RewardHint
useSessionPersistence	Hook	hooks/useSessionPersistence.ts	StudyView.tsx	services/studySessionPersistence, db/queries/sessions	PersistedStudySession
useSyncRuntime / useServerHeartbeat / useServiceWorkerConfig	Hook	hooks/useSyncRuntime.ts / useServerHeartbeat.ts / useServiceWorkerConfig.ts	AppInitializer, HomeView	syncCoordinator, syncReachability, syncConfig	SyncReachability
useAlgorithmMigration / useBacklogSmoother / useAppBadge	Hook	hooks/*.ts	AppInitializer.tsx	services/algorithmMigration, db/queries	CardRecord

ELEMENT	TYP	DEFINIERT IN	VERWENDET VON	ABHÄNGIGKEITEN	DATENOBJEKTE
useStreak / useHeatmap	Hook	hooks/useStreak.ts / useHeatmap.ts	DailyGoalRing, StreakBadge, ReviewHeatmap	db (reviews), utils/workers/statsWorkerClient	ReviewRecord
useLocalMesserVideos / useVideoSource	Hook	hooks/useLocalMesserVideos.ts	VideosView.tsx	db/queries/videoDownloads, utils/localVideoManifest, videoDownloadQueue	VideoDownloadRecord, VideoBlobRecord
useMesserVideoProgress / useVideoRecallScores	Hook	hooks/useMesserVideoProgress.ts / useVideoRecallScores.ts	VideosView, VideoRecallCheck, useTodayPackage, useLearningUnits	localStorage direkt	MesserVideoProgress, RecallScoreMap
useWebPushSubscription / usePwinstall / useFullscreen / useWakeLock / useVisualViewport / useHandsetLayout	Hook	hooks/*.ts	AppInitializer, HomeView, App.tsx, StudyView, VideosView	services/webPush, env.ts, Browser-APIs	—
useFloatingMenu / useToast / useDayStartMs / useGlobalErrorLogging / useStoragePersistence / useAutoJoinDefaultProfile	Hook	hooks/*.ts	DeckCard, HomeDeckToolbar, ToastContainer, App.tsx, AppInitializer	services/errorLog, profileService, syncConfig, syncCoordinator	—

16.5 Services

ELEMENT	TYP	DEFINIERT IN	VERWENDET VON	ABHÄNGIGKEITEN	DATENOBJEKTE	BESCHREIBUNG
syncConfig	Service	services/syncConfig.ts	alle übrigen Sync-Services, ProfileSyncSection, SettingsContext	constants/applidentity, utils/id	SyncConfig	Endpoint-/Token-Cache, fetchWithTimeout, Auth-Header
syncQueue	Service	services/syncQueue.ts	db/queries/* (nach jeder Mutation), OfflineStatusBanner	SyncQueueDB (Dexie), profileService, deckContentScope, syncConfig	SyncQueueRecord	Durable Push-Queue mit Retry/Backoff/Dead-Letter
syncPull	Service	services/syncPull.ts	syncCoordinator.ts	normalize/*, sync/operationResolver, workers/sync-applier, workers/snapshot-normalizer, deckContentScope	alle Dexie-Record-Typen	Handshake/Snapshot/DePull/Apply, 1399 Zeilen
syncCoordinator	Service	services/syncCoordinator.ts	ProfileSyncSection, useAutoJoinDefaultProfile, useSyncRuntime	syncQueue, syncPull, syncReachability	—	Serialisiert Push+Pull, Trigger-Runtime (Intervall/online/SW)
syncReachability	Service	services/syncReachability.ts	syncCoordinator, ProfileSyncSection, useServerHeartbeat	syncConfig	SyncReachabilityState	Health-Check mit exponentiellem Backoff + SW-Heartbeat-Delegation
syncedDeckScope	Service	services/syncedDeckScope.ts	useHomeDerivedData.ts, shuffleSession.ts	db, syncConfig, profileService, utils/reviewDecks	Deck-IDs	Bestimmt sync-/studien-relevante Deck-Menge
studySessionReducer	Service (Reducer)	services/studySessionReducer.ts	StudyView, ShuffleStudyView, useSessionPersistence	sessionRecovery, studySessionPersistence (Typ)	SessionState, Card	Redux-artiger Session-State-Reducer, 9 Actions
sessionRecovery	Service	services/sessionRecovery.ts	studySessionReducer.ts	—	RecoveryState	Again/Hard-Practice-Zähl + Requeue-Entscheidung
studyCardOrdering	Service	services/studyCardOrdering.ts	StudyView, useLearningUnits, useTodayPackage, db/queries/decks	cardOrdering, utils/time	Card	Fällig-Filter, Interleaving, Tages-Kartenauswahl
cardOrdering	Service	services/cardOrdering.ts	studyCardOrdering, shuffleSession (intern)	utils/hash, utils/time	Card	Geteilte Prioritäts-/Vergleichs-Hell
studyModeSelector	Service	services/studyModeSelector.ts	StudyView, ShuffleStudyView	utils/cardVariant, utils/hash	Card	Seeded Auswahl der Dra Match-Karten pro Session
studySessionPersistence	Service	services/studySessionPersistence.ts	StudyView, ShuffleStudyView, SettingsContext, db/queries/sessions	utils/time	PersistedStudySession	Versionierte (De-)Serialisierung der Session
shuffleSession	Service	services/shuffleSession.ts	HomeShuffleCollectionModal, useHomeDerivedData, useShuffleCards	db/queries/decks, cardOrdering, studyCardOrdering, syncedDeckScope	ShuffleStudyCard	Kartenpool über mehrere Decks + Gewichtung
profileService	Service	services/profileService.ts	ProfileSyncSection, SettingsContext, useAutoJoinDefaultProfile, LabsView, LearningUnitsView, VideosView	db, utils/id, syncConfig, errorLog	ProfileRecord	Geräte-/Profil-Identität + kompletter /auth/*-Client
deckHierarchy	Service	services/deckHierarchy.ts	AppInitializer.tsx	db, syncQueue, securityDeckHierarchy	DeckRecord	Erzeugt/aktualisiert feste SY0-701-Deck-Hierarchie beim Start

ELEMENT	TYP	DEFINIERT IN	VERWENDET VON	ABHÄNGIGKEITEN	DATENOBJEKTE	BESCHREIBUNG
learningUnitRunner	Service	services/learningUnitRunner.ts	LabsView, LearningUnitsView, VideosView, useLearningUnits	utils/learningUnits, studyCardOrdering, db/queries/learningUnits, labSnapshot	LearningUnitExecution, LabAttemptRecord	Orchestriert Start/Fortschritt der Lerneinheiten, schreibt Reviews/XP
learningCoach	Service	services/learningCoach.ts	SessionCoachPanel, StudyView, ShuffleStudyView	—	SessionReviewEvent	Rein beratende Post-Session-Zusammenfassung, keine Persistenz
fsrsOptimizer	Service	services/fsrsOptimizer.ts	SettingsModal.tsx	db (reviews), @open-spaced-repetition/binding (WASM)	ReviewRecord	Client-seitige FSRS-Parameteroptimierung
algorithmMigration	Service	services/algorithmMigration.ts	useAlgorithmMigration.ts	db, utils/sm2, utils/algorithmParams, syncQueue	CardRecord	Reversible SM2 → FSRS-Migration, batchweise
brandMigration	Service	services/brandMigration.ts	main.tsx	dexie (separate Legacy-DB-Klassen), constants/applidentity	—	Einmalige Rebrand-Migration (anki-pwa-* → card-pwa-*)
webPush	Service	services/webPush.ts	SettingsModal, useHomeController, useWebPushSubscription	syncConfig	PushSubscriptionJSON	Web-Push-Abonnement-Registrierung
offlineStatus	Service (Singleton)	services/offlineStatus.ts	OfflineStatusBanner.tsx	env.ts	OnlineStatus	Online/Offline/Slow-Erkennung mit SubscribeListe
errorLog	Service	services/errorLog.ts	AppErrorBoundary, SettingsModal, useGlobalErrorLogging, profileService, syncQueue, syncPull	constants/applidentity	ErrorLogEntry	Client-Fehlerprotokoll (localStorage, Ringpuffer 500)
updateService	Service	services/updateService.ts	KEIN LAUFZEIT-VERWENDER	env.ts	—	SW-Update-Check — s. §14, wirkt abgelöst

16.6 DB & Queries

ELEMENT	TYP	DEFINIERT IN	VERWENDET VON	ABHÄNGIGKEITEN	DATENOBJEKTE	BESCHREIBUNG
CardPwaDB (Klasse db)	Dexie-DB	db/index.ts	alle db/queries/*, viele Hooks direkt (liveQuery)	dexie	alle Kern-Records	Hauptdatenbank, alle Tabellen
LearningUnitsDB (Klasse LearningUnitsDb)	Dexie-DB	db/learningUnitsDb.ts	db/queries/learningUnits.ts	dexie	LearningUnit-Familie	Additive, komplett
SyncQueueDB (Klasse syncDb)	Dexie-DB	services/syncQueue.ts	syncQueue.ts intern	dexie	SyncQueueRecord	Dritte, unabhängig
db/queries/decks.ts	Query-Modul	db/queries/decks.ts	HomeView, App.tsx, useHomeDerivedData, useTagCardIndex, useTodayPackage, sharedReads-Konsumenten	db, utils/tagIdentity, services/studyCardOrdering, sharedReads	Deck, Card	Deck-Baum, Karte createDeck/deleteDeck
db/queries/cards.ts	Query-Modul	db/queries/cards.ts	CardFormModal.tsx	db, syncQueue (enqueue)	CardRecord	createCard/updateCard, normalizeDueDate
db/queries/reviews.ts	Query-Modul	db/queries/reviews.ts	StudyView, ShuffleStudyView, SettingsModal, useAppBadge, useBacklogSmoother	db, sharedReads, syncQueue (enqueue+outbox), workers/statsWorkerClient, backlogSmoother	ReviewRecord, CardRecord	recordReview/createReview, undoReview, forgetReview
db/queries/sessions.ts	Query-Modul	db/queries/sessions.ts	StudyView, ShuffleStudyView, App.tsx, useSessionPersistence	db, utils STORAGE_KEYS	ActiveSessionRecord	IndexedDB-Session (localStorage ab)
db/queries/gamification.ts	Query-Modul	db/queries/gamification.ts	useGamificationProfile, useSessionRewards	db, utils/gamification	ReviewRecord	Leitet XP/Streak/Coins ab
db/queries/shuffleCollections.ts	Query-Modul	db/queries/shuffleCollections.ts	HomeShuffleCollectionModal, useHomeController, useCardDb	db, syncQueue (enqueue)	ShuffleCollectionRecord	CRUD für Shuffle-Collections
db/queries/videoDownloads.ts	Query-Modul	db/queries/videoDownloads.ts	useLocalMesserVideos.ts	db	VideoDownloadRecord, VideoBlobRecord	Offline-Video-Speicherung synchronisiert
db/queries/videoNotes.ts	Query-Modul	db/queries/videoNotes.ts	useVideoNotes.ts, VideoNotesPanel.tsx	db, syncQueue (enqueue), videoTagMeta, utils/videoTags/Links	VideoNoteRecord	Profil-gespeicherte Notizen abgeleitet
db/queries/videoTagMeta.ts	Query-Modul	db/queries/videoTagMeta.ts	useVideoTags.ts	db, decks.ts (listAllCards)	VideoTagMetaRecord	Tag-Metadaten (LocalStorage, kein Sync)
db/queries/diagnostics.ts	Query-Modul	db/queries/diagnostics.ts	SettingsModal.tsx, reviews.ts (intern)	localStorage direkt, db (dev-only)	AlgorithmDiagnosticsEntry	Dev-Verifikation der Algorithmen
db/queries/answerStats.ts	Query-Modul (nicht im Barrel)	db/queries/answerStats.ts	useLearningUnits.ts, learningUnitRunner.ts	db, decks.ts, utils/learningUnits, utils/answerStats	ReviewRecord, AnswerStats	Antwortstatistik für Empfehlungen

ELEMENT	TYP	DEFINIERT IN	VERWENDET VON	ABHÄNGIGKEITEN	DATENOBJEKTE	BESCHREIBUNG
db/queries/sharedReads.ts	Query-Modul (nicht im Barrel)	db/queries/sharedReads.ts	decks.ts, reviews.ts (intern)	db	CardRecord, DeckRecord	In-Flight-Read-Compensation
db/queries/learningUnits.ts	Query-Modul (nicht im Barrel)	db/queries/learningUnits.ts	useLearningUnits, learningUnitRunner, LearningUnitsView, VideosView, SettingsContext, syncPull, dbBackup	learningUnitsDb (Dexie)	LearningUnitExecution, VideoProgressRecord, LabAttemptRecord	Vollständige Persönliche Lerneinheiten-Sync

16.7 Utils — Algorithmen & Text-Parsing

ELEMENT	TYP	DEFINIERT IN	VERWENDET VON	ABHÄNGIGKEITEN	DATENOBJEKTE	BESCHREIBUNG
fsrs.ts	Algorithmus	utils/fsrs.ts	db/queries/reviews.ts	ts-fsrs (npm), algorithmParams	CardRecord	FSRS-Adapter auf einheitliches Card-Feld-Set
sm2.ts	Algorithmus	utils/sm2.ts	CardFormModal, algorithmMigration, db/queries/*, utils/import/*	algorithmParams	CardRecord	Reines SM-2, plus zentrale Typ-/Queue-Konstanten
algorithmParams.ts	Utils	utils/algorithmParams.ts	fsrs.ts, sm2.ts, algorithmMigration, SettingsContext, db/queries/*	—	SM2Params, FSRSPParams	Parameter-Normalisierung + Ease ↔ Difficulty-Brücke
cardTextParser.ts	Utils	utils/cardTextParser.ts	sehr breit: CardFace, alle Karten-Renderer, DeckCard, CardFormModal, VideoRecallCheck u. v. a.	—	Question, Answer	Zentrale Front/Back-Text → Struktur-Übersetzung aller Kartentypen
cardVariant.ts	Utils	utils/cardVariant.ts	CardFormModal, CardFace, FreeRecallCard, studyModeSelector, HomeDeckCardsModal	cardTextParser	Card	Erkennt Kartenform (standard/mc/ordering/matching/drag-match/free-recall)
pbqScoring.ts	Utils	utils/pbqScoring.ts	MatchingCard, OrderingCard, LabScenarioView, labSnapshot	—	—	Reines Scoring für Ordering/Matching/Decision
dragMatchScoring.ts / freeRecallScoring.ts	Utils	utils/dragMatchScoring.ts / freeRecallScoring.ts	DragMatchCard / FreeRecallCard	cardTextParser	Answer	Trivialer Score für die jeweilige Übungsform
hash.ts	Utils	utils/hash.ts	Karten-Renderer, cardOrdering, studyModeSelector, labSnapshot	—	—	FNv-1a-Hash + seeded Fisher-Yates-Shuffle (deterministische Optionsreihenfolge)
id.ts	Utils	utils/id.ts	CardFormModal, profileService, syncConfig, db/queries/decks, shuffleCollections, import/csvImporter	—	—	UUIDv7-Generator
time.ts	Utils	utils/time.ts	sehr breit: useDayStartMs, studyCardOrdering, db/queries/*, stats/aggregate, gamification	—	—	Fälligkeits-/Tagesgrenzen-Basis (resolveDueAtMs, getDayStartMs)
gamification.ts (utils)	Utils	utils/gamification.ts	useSessionRewards, db/queries/gamification	types, time.ts	ReviewRecord	XP/Level/Streak/Quest-Berechnung

16.8 Utils — Normalisierung, Sync-Konflikte, Web Worker, Import-Pipeline

ELEMENT	TYP	DEFINIERT IN	VERWENDET VON	ABHÄNGIGKEITEN	DATENOBJEKTE	BESCHREIBUNG
normalize/card.ts, deck.ts, review.ts, shuffleCollection.ts, videoNote.ts	Utils (5 Dateien)	utils/normalize/*.ts	operationResolver, syncPull, normalize/snapshot.ts	db (Typen)	CardRecord, DeckRecord, ReviewRecord, ShuffleCollectionRecord, VideoNoteRecord	Tolerante Validierung, eingehende Payloads
normalize/snapshot.ts	Utils	utils/normalize/snapshot.ts	workers/snapshot-normalizer.worker.ts, syncPull	alle 5 normalize/*-Dateien	SnapshotNormalizeResult	Batch-Norm eines voller Snapshots

ELEMENT	TYP	DEFINIERT IN	VERWENDET VON	ABHÄNGIGKEITEN	DATENOBJEKTE	BESCHR
sync/operationResolver.ts	Utils	utils/sync/operationResolver.ts	workers/sync-applier.worker.ts, syncPull.ts	normalize/*, services/syncQueue (Typ)	OperationDiff	Reines Konfliktauflösung Regelwerk first/LWW/first
workers/workerPool.ts	Worker-Infrastruktur	utils/workers/workerPool.ts	statsWorkerClient, apkgImporter, syncPull	Browser Worker/MessageChannel	—	Generische Fabrik mit Fallback
workers/apkg-parser.worker.ts	Worker	utils/workers/apkg-parser.worker.ts	utils/import/apkgImporter.ts	import/apkgParserCore	ParsedImport	Entlagert. a Parsing vor
workers/csv-import.worker.ts	Worker	utils/workers/csv-import.worker.ts	utils/import/csvImporter.ts	import/csvImporter (parseCsvText)	ParsedImport	Nur ab 500 Dateigröße
workers/snapshot-normalizer.worker.ts	Worker	utils/workers/snapshot-normalizer.worker.ts	services/syncPull.ts	normalize/snapshot	SnapshotNormalizeResult	Worker-Wrappers Snapshot-N
workers/sync-applier.worker.ts	Worker	utils/workers/sync-applier.worker.ts	services/syncPull.ts	sync/operationResolver	OperationDiff	Worker-Wrappers Batch-Konf
workers/stats.worker.ts + statsWorkerClient.ts	Worker + Client	utils/workers/stats.worker.ts / statsWorkerClient.ts	useStreak, useHeatmap, db/queries/reviews (Forecast)	utils/stats/aggregate	HeatmapBucket, StreakStats	Heatmap/S mit Memois Cache im V
stats/aggregate.ts	Utils	utils/stats/aggregate.ts	stats.worker.ts, statsWorkerClient.ts (Fallback)	utils/time	ReviewRecord, CardRecord	Reiner Agg (auch sync)
import/importPipeline.ts	Utils	utils/import/importPipeline.ts	components/ImportModal.tsx	db, services/syncQueue, import/types	ImportPlan, ImportResult	Diff/Plan/Export Chunks à 2
import/apkgImporter.ts + apkgParserCore.ts + ankiDatabase.ts + ankiFields.ts + mcNormalizer.ts + htmlStrip.ts	Utils (6 Dateien)	utils/import/*.ts	ImportModal (via apkgImporter), apkg-parser.worker	JSZip, sql.js, vendor/fzstd, utils/sm2	ImportedCard, ImportedDeck	Vollständige Anki-.apkg/Import inkl. Dekompres
import/csvImporter.ts	Utils	utils/import/csvImporter.ts	ImportModal, csv-import.worker	papaparse, utils/sm2, utils/dbBackup, mcNormalizer, utils/fid	ImportedCard, ImportedDeck	Anki-TXT/CS Backup-CS Backup-TX
import/jsonBackupImporter.ts	Utils	utils/import/jsonBackupImporter.ts	components/ImportModal.tsx	utils/dbBackup (Typ)	DbBackupPayload	Parst eigen Vollbackup

16.9 Utils — Domänenlogik (Lerneinheiten, Labs, Video-Tags, Sonstiges)

ELEMENT	TYP	DEFINIERT IN	VERWENDET VON	ABHÄNGIGKEITEN
learningUnits.ts	Utils (Domänenkern)	utils/learningUnits.ts	LearningUnitsView, HomeLearningUnits*, useLearningUnits, learningUnitRunner, db/queries/learningUnits, learningPlanMapping, learningUnitRanking	types, localVideoManifest, SERVICES/STUDYCARDORDERING
learningUnitRanking.ts	Utils	utils/learningUnitRanking.ts	LearningUnitsView, LearningPlanPanel, HomeLearningUnits*, useLearningUnits	learningUnits.ts (Typen), examDate
learningPlanMapping.ts	Utils	utils/learningPlanMapping.ts	LearningPlanPanel, useLearningUnits	learningUnits.ts, securityDeckHierarchy
securityDeckHierarchy.ts	Utils	utils/securityDeckHierarchy.ts	sehr breit: LearningUnitsView, HomeView, ShuffleStudyView, VideosView, LabsView, deckHierarchy, learningUnitRunner	—
todayPackage.ts	Utils	utils/todayPackage.ts	App.tsx, useTodayPackage, useLearningUnits, learningUnitRunner, HomeView	localVideoManifest, useVideoRecallScores (Typ), examDate
labGenerator.ts / labSnapshot.ts / labProgress.ts / labTraining.ts	Utils (4 Dateien)	utils/lab*.ts	labSnapshot: learningUnitRunner; labProgress: LabsView, useLearningUnits; LABGENERATOR/LABTRAINING: NUR TESTS	data/labBlueprints, data/labScenarios, hash.ts, pbqScoring
tagIdentity.ts	Utils	utils/tagIdentity.ts	sehr breit: VideoTagSidebar, db/queries/decks, videoTagMeta, videoNotes, useTagCardIndex, dbBackup	—
videoTags.ts / videoLinks.ts / videoTimeAnchors.ts	Utils	utils/videoTags.ts / videoLinks.ts / videoTimeAnchors.ts	VideoNotesPanel, db/queries/videoNotes, videoTagPageData, videoTagStats	tagIdentity
videoTagStats.ts / videoTagPageData.ts / videoNoteSignals.ts	Utils	utils/*.ts	VideoTagSidebar, TagCollectionPanel, useVideoTags, db/queries/videoTagMeta	tagIdentity, videoTimeAnchors
tagSuggestions.ts	Utils	utils/tagSuggestions.ts	VideoNotesPanel.tsx	tagIdentity
videoPlayback.ts / videoDownloadQueue.ts / localVideoManifest.ts	Utils	utils/*.ts	MesserVideoPlayer, useLocalMesserVideos, VideosView, useTodayPackage, useLearningUnits	—

ELEMENT	TYP	DEFINIERT IN	VERWENDET VON	ABHÄNGIGKEITEN
reviewDecks.ts / deckContentScope.ts	Utils	utils/reviewDecks.ts / deckContentScope.ts	HomeView, ProfileSyncSection, syncedDeckScope, syncQueue, syncPull	constants/applIdentity
examDate.ts	Utils	utils/examDate.ts	LearningPlanPanel, SettingsContext, todayPackage, learningUnitRanking	—
dbBackup.ts	Utils	utils/dbBackup.ts	ImportModal, SettingsModal, useHomeDerivedData, useHomeController, csvImporter	db, db/queries/learningUnits, videoTags, tagIdentity, videoNotes (normalizeTags)
backlogSmoother.ts / gamification.ts / buildInfo.ts / acronymQuiz.ts / answerStats.ts (utils) / motivationQuote.ts	Utils	utils/*.ts	db/queries/reviews (backlogSmoother), db/queries/gamification, SettingsModal, AcronymPracticeView, db/queries/answerStats, App.tsx	meist keine internen Abhängigkeiten

16.10 Statische Daten, Konstanten, Typen

ELEMENT	TYP	DEFINIERT IN	VERWENDET VON	ABHÄNGIGKEITEN
SY0701_REQUIREMENTS_MANIFEST	Statische Daten	data/sy0701Requirements.ts	LearningUnitsView.tsx	—
SY0701_CONTENT_MAP	Statische Daten	data/sy0701ContentMap.ts	useLearningUnits.ts, learningUnitRunner.ts	—
SY0701_ACRONYMS	Statische Daten	data/sy0701Acronyms.ts	utils/acronymQuiz, AcronymPracticeView	—
ACRONYM_DEFINITIONS	Statische Daten (neu, unversioniert)	data/acronymDefinitions.ts	AcronymPracticeView.tsx	—
LAB_SCENARIOS / LAB_CATEGORIES	Statische Daten	data/labScenarios.ts	LabsView, LabScenarioView, labUi, useLearningUnits, learningUnitRunner, labGenerator, labSnapshot	—
LAB_BLUEPRINTS	Statische Daten	data/labBlueprints.ts	NUR UTILS/LABGENERATOR.TS (UNGENUTZT)	data/labScenarios (Typ)
MESSER_TRANSCRIPT_QUESTIONS / MESSER_VIDEO_BY_QUESTION_ID	Statische Daten	data/messerTranscriptQuestions.ts / messerVideoQuestionMap.ts	VideoRecallCheck.tsx	—
MOTIVATION_QUOTES	Statische Daten	data/motivationQuotes.ts	utils/motivationQuote.ts	—
AI_MODE_GUIDES	Statische Daten	data/aiModeGuides.ts	NUR EIGENER TEST	—
Card, Deck, Settings-Typen, View, Rating u. a.	Typen	types/index.ts	praktisch alle Views/Komponenten/Hooks/Services	db (Re-Export einzelner DB-Typen)
STORAGE_KEYS, SW_CHANNELS, DATABASE_NAMES, REVIEW_UPDATED_EVENT u. a.	Konstanten	constants/appIdentity.ts	praktisch alle Schichten	—
UI_TOKENS	Konstanten	constants/ui.ts	viele Komponenten (Modals, Bars, RatingBar)	—

S17 Datenobjekt-Dokumentation

Für jedes zentrale Datenobjekt: Definition, Erzeugung, Lese-/Schreibzugriffe, Speicherort, Übertragungsweg, Seiteneffekte.

Card / CardRecord

Definiert in	db/index.ts (CardRecord) · types/index.ts (Card, App-facing)
Erzeugt in	db/queries/cards.ts → createCard()
Gelesen von	CardFace + alle Karten-Renderer, StudyView/ShuffleStudyView, DeckCard, HomeDeckCardsModal, fast alle Hooks unter hoo
Verändert von	db/queries/cards.ts (updateCard), db/queries/reviews.ts (recordReview/undoReview/resetLearningProgress/s
Gespeichert in	Dexie db.cards (CardPwaDB)
Übertragen via	Props (Karten-Renderer), Rückgabewert von db/queries, Sync-Operationen card.create/update/delete/schedule.f
Seiteneffekte bei Änderung	REVIEW_UPDATED_EVENT wird gefeuert → alle liveQuery-Hooks aktualisieren; bei recordReview zusätzlich Schreiben in d drainTransactionalOutbox → SyncQueueDB → Server-Push

Deck / DeckRecord

Definiert in	db/index.ts (DeckRecord) · types/index.ts (Deck, mit rekursiven subDecks)
Erzeugt in	db/queries/decks.ts → createDeck() · services/deckHierarchy.ts (feste SY0-701-Struktur beim Start) · db/c
Gelesen von	HomeView, DeckCard, alle Study-Views, syncedDeckScope, learningPlanMapping
Verändert von	db/queries/decks.ts (deleteDeck – kaskadiert auf cards/reviews)
Gespeichert in	Dexie db.decks
Übertragen via	Sync-Operationen deck.create/delete, Props (Deck-Baum an DeckCard/HomeDeckListSection)
Seiteneffekte bei Änderung	Deck-Löschung ist kaskadierend (Reviews hart gelöscht, Karten/Deck soft-deleted) und triggert eigene Sync-Operation mit vo

ReviewRecord

Definiert in	db/index.ts
Erzeugt in	db/queries/reviews.ts → recordReview() (innerhalb Dexie-Transaktion mit cards+syncOutbox)
Gelesen von	useStreak, useHeatmap, useGamificationProfile, DeckMetricsModal, ShuffleMetricsModal, fsrsOptimizer, learningCoach (indirekt via SessionReviewEvent)
Verändert von	db/queries/reviews.ts → undoReview() (Löschung), resetLearningProgress() (db.reviews.clear())
Gespeichert in	Dexie db.reviews
Übertragen via	Sync-Operationen review/review.undo; Worker-Payload an stats.worker.ts für Aggregation
Seiteneffekte bei Änderung	Ist die alleinige Quelle für Gamification/Streak/Heatmap — jede Änderung invalidiert diese Ableitungen über REVIEW_UPDATED_EVENT; beim Pull gilt „Karte muss lokal existieren“, sonst wird die Review verworfen (keine verwaisten Reviews)

ShuffleCollectionRecord

Definiert in	db/index.ts
Erzeugt in	db/queries/shuffleCollections.ts → createShuffleCollection()
Gelesen von	HomeShuffleSection, ShuffleStudyView (via useShuffleCards → shuffleSession), useHomeDerivedData
Verändert von	db/queries/shuffleCollections.ts → updateShuffleCollection(), deleteShuffleCollection() (soft-delete)
Gespeichert in	Dexie db.shuffleCollections
Übertragen via	Sync-Operationen shuffleCollection.upsert/delete; deckIds ist nur eine lose Referenz, Karten werden nie kopiert
Seiteneffekte bei Änderung	Kartenpool wird bei jeder Session neu aus den referenzierten Decks gebaut (services/shuffleSession.ts) — keine denormalisierte Kopie, daher immer konsistent mit dem aktuellen Deck-Zustand

Settings (nicht Dexie)

Definiert in	contexts/SettingsContext.tsx
Erzeugt in	DEFAULT_SETTINGS-Konstante, gemerged mit persistiertem JSON beim Hydrieren
Gelesen von	praktisch alle Views (Sprache, Algorithmus, Limits, Theme-Font)
Verändert von	~25 set*-Funktionen aus useSettings(), aufgerufen v. a. aus SettingsModal
Gespeichert in	localStorage-Key card-pwa-settings (JSON)
Übertragen via	Nur examDateIso/examDateUpdatedAt werden zusätzlich als Sync-Operation examDate.upsert übertragen — einziges Settings-Feld mit Server-Sync, Muster für künftige Fälle laut Projektkontext

Seiteneffekte bei Änderung	Änderung von <code>fontFamily</code> setzt <code>CSS-Custom-Properties</code> auf <code>document.documentElement</code> ; <code>examDateIso</code> -Änderung schreibt zusätzlich in <code>db/queries/learningUnits.ts</code> (<code>saveDraftLearnerExamPlan</code>) und triggert <code>REVIEW_UPDATED_EVENT</code> -artige Neuberechnung der Lerneinheiten-Pacing
----------------------------	--

ProfileRecord

Definiert in	<code>db/index.ts</code>
Erzeugt in	<code>services/profileService.ts</code> → <code>buildLocalProfile()</code> (lokal) bzw. nach <code>createServerProfile()/joinPublicProfile()</code>
Gelesen von	<code>SettingsContext</code> (Hydratation beim Start), <code>ProfileSyncSection</code> , <code>syncConfig</code> (Cache), <code>syncedDeckScope</code>
Verändert von	<code>SettingsContext.setProfile()</code> , <code>profileService.saveProfile()/clearProfile()</code>
Gespeichert in	Dexie <code>db.profile</code> (PK <code>'current'</code>), zusätzlich Cookie <code>card_pwa_profile_hint</code>
Übertragen via	Nicht selbst als Sync-Operation — steuert stattdessen, welcher Server-Endpoint/Token für alle anderen Sync-Aufrufe verwendet wird
Seiteneffekte bei Änderung	Profilwechsel löst <code>SYNC_RUNTIME_CONFIG_CHANGED_EVENT</code> aus (Sync-Runtime neu konfiguriert) und in <code>ProfileSyncSection</code> Snapshot/Rollback-Mechanismus (<code>snapshotLocalStudyDataForRollback</code>) für den Fall eines fehlschlagenden Profilwechsels

VideoNoteRecord / VideoTagMetaRecord

Definiert in	<code>db/index.ts</code>
Erzeugt/verändert in	<code>db/queries/videoNotes.ts</code> → <code>saveVideoNote()</code> (Upsert oder Delete bei leerem Inhalt), ruft intern <code>videoTagMetaRecord</code> an
Gelesen von	<code>VideoNotesPanel</code> , <code>TagCollectionPanel</code> , <code>VideoTagSidebar</code> (über die jeweiligen <code>liveQuery</code> -Hooks)
Gespeichert in	Dexie <code>db.videoNotes2</code> (PK [<code>profileId</code> , <code>objective</code>]) bzw. <code>db.videoTagMeta</code> (PK [<code>profileId</code> , <code>tagId</code>])
Übertragen via	<code>VideoNoteRecord</code> : Sync-Operation <code>videoNote.upsert/delete</code> . <code>VideoTagMetaRecord</code> : kein Sync — bewusst lokal, Notiztext wird über <code>videoNote</code> übertragen
Seiteneffekte bei Änderung	Tags werden nicht separat eingegeben, sondern aus <code>#tag-Inline</code> -Syntax im Notiztext abgeleitet (<code>extractTags</code>) — jede Textänderung kann Tags verändern

SyncQueueRecord (Operations-Log)

Definiert in	<code>services/syncQueue.ts</code>
Erzeugt in	<code>enqueueSyncOperation()</code> , aufgerufen aus praktisch jeder mutierenden Funktion in <code>db/queries/*</code> ; bei <code>recordReview</code> zusätzlich indirekt über <code>db.syncOutbox + drainTransactionalOutbox()</code>
Gelesen/verändert von	<code>services/syncQueue.ts</code> → <code>flushSyncQueue()</code> (Push, Retry/Backoff, Dead-Letter nach 20 Versuchen)
Gespeichert in	Eigene Dexie-Datenbank <code>SyncQueueDB</code> (Tabelle <code>queue</code>) — bewusst getrennt von den Business-Daten
Übertragen via	POST an den Sync-Server, Header <code>X-Idempotency-Key: opId</code>
Seiteneffekte bei Änderung	Vor dem Senden greift eine Deck-Scope-Filterung (§13: dreifach implementiert) — Operationen außerhalb des gewählten Sync-Umfangs werden „deferred“ (<code>nextRetryAt = MAX_SAFE_INTEGER</code>) statt gelöscht, reaktivierbar via <code>wakeDeferredSyncQueue()</code>

Lerneinheiten-Familie (LearningUnitsDB-Records)

Definiert in	<code>db/learningUnitsDb.ts</code>
Erzeugt/verändert in	<code>db/queries/learningUnits.ts</code> , orchestriert von <code>services/learningUnitRunner.ts</code>
Gelesen von	<code>LearningUnitsView</code> , <code>HomeLearningUnits*</code> , <code>useLearningUnits</code>
Gespeichert in	Eigene Dexie-Datenbank <code>LearningUnitsDB</code> , alle Keys beginnen mit <code>profileId</code>
Übertragen via	Kein Sync — Server-Synchronisation für dieses System ist laut Code-Kommentar für eine spätere Phase vorgesehen, aktuell rein lokal
Seiteneffekte bei Änderung	<code>LearningUnitExecution</code> und <code>LabAttemptRecord</code> „frieren“ ihren fachlichen Inhalt (Content-Map-Referenzen bzw. Szenario-Snapshot) beim Start ein — spätere Änderungen an <code>data/sy0701ContentMap.ts</code> oder <code>data/labScenarios.ts</code> wirken sich nicht rückwirkend auf laufende/abgeschlossene Executions aus; <code>resetProfileLearningEvidence()</code> arbeitet append-only (erhöht eine <code>evidenceEpoch</code> , alte Einträge bleiben bestehen, zählen aber nicht mehr als Evidenz)

§18 Anhang: Unklare oder dynamisch erzeugte Abhängigkeiten

Punkte, die sich nicht abschließend aus statischer Quellcode-Analyse bestimmen lassen

— explizit als solche gekennzeichnet, wie gefordert.

- **LAUFZEITABHÄNGIG** `window.launchQueue File-Handler` (App.tsx) — welche Datei/App den Consumer auslöst, hängt von Browser-/OS-Dateizuordnung ab, nicht aus dem Quellcode allein ableitbar.
- **BUILD-ABHÄNGIG** **Service-Worker-Precache-Asset-Graph** — `public/service-worker.js` scannt zur Install-Zeit `index.html` und referenzierte JS/CSS/WASM-Bundles per Regex, um die zu cachenden Dateien zu bestimmen (`MAX_PRECACHE_GRAPH_ASSETS = 120`). Die exakte Dateiliste ist erst nach dem Vite-Build bekannt, nicht aus `src/` allein.
- **DYNAMISCHE IMPORTS (CODE-SPLITTING)** Mehrere Module werden erst zur Laufzeit nachgeladen und tauchen daher nicht im statischen Import-Graph auf: `ui/motionFeatures.ts` (framer-motion domMax-Bundle), `db/queries/learningUnits.ts` (nur innerhalb `resolveSessionDeckName` in App.tsx), `data/messerTranscriptQuestions.ts` (in `VideoRecallCheck`), `utils/import/apkgParserCore.ts` (Main-Thread-Fallback, wenn Worker fehlschlägt), `sql.js/JSZip` (innerhalb `ankiDatabase.ts/apkgParserCore.ts`).
- **BROWSER-ABHÄNGIGE FEATURE-ERKENNUNG** `services/offlineStatus.ts` liest `navigator.connection` (Network Information API) für die „slow“-Erkennung — nur in Chromium-Browsern verfügbar; das Verhalten in anderen Browsern ist laufzeit-, nicht quellcodeabhängig.
- **WORKER-NACHRICHTENGRENZE** Die `postMessage`-Payloads zwischen Main-Thread und den fünf Web Workern (§16.8) sind strukturell typisiert, aber TypeScript prüft sie *nicht* über die Thread-Grenze hinweg — Konsistenz zwischen Send- und Empfangsseite ist Konvention, kein Compiler-Garant.
- **NICHT EINDEUTIG VERORTBAR** `components/MetaBalls.tsx` — keine Einbindungsstelle in den untersuchten Kern-Komponenten gefunden (siehe §14); ob und wo sie aktuell gerendert wird, war aus dem durchsuchten Code nicht abschließend bestimmbar.
- **SERVER-SEITIG, AUSSERHALB DIESES SCOPES** Das Sync-Server-Backend (`sync_operations-Op-Log`, Konfliktbehandlung serverseitig, PIN-Auth) liegt außerhalb von `card_pwa/` und wurde nicht analysiert — alle Aussagen zum Server basieren ausschließlich auf dem, was der Client-Code über die Endpunkte und Payload-Shapes preisgibt.

Erstellt durch Quellcode-Analyse von `card_pwa/src` (7 parallele Recherche-Durchläufe) sowie `public/service-worker.js`. Alle Modul-/Funktionsnamen und Dateipfade sind Zitate aus dem Repository; Beziehungen wurden nicht erfunden, unklare Fälle sind explizit markiert. Stand: Branch `branch_01`.